Computer Architectures notes

Luca Seggiani
2026

1 Lesson 22-09-26

1.1 Introduction

The aim of this course is to provide basic knowledge about the architecture of modern *microprocessor*-based systems. Special emphasis will be put on the *microprocessor core* itself.

The course is structured in two main blocks:

1. Modern processor architecture. This wil cover computer design, the *RISC* processor architecture, superscalar processors and disrupting architectures;
2. *ARM*-ased systems, that is ARM assembly language and ARM *Cortex-M3* based systems. Both development boards and emulation systems will be used to aid in the study of ARM-based systems.

1.2 CISC vs. RISC

We first go over what an **ISA** (*Instruction Set Architecture*) is in the context of processor design. Then we distinguish between **CISC** (*Complex Instruction Set Computer*) and **RISC** (*Reduced Instruction Set Computer*) architectures.

1.2.1 High level vs. Low level languages

Many programming languages exist. They can be classified into:

- **High-level** programming languages, which abstract over the machine to offer more sophisticated capabilities. This gives developers the freedom to express the same algorithm in many ways, and usually to express the same algorithm with significantly less lines of code than those used in low-level languages;
- **Low-level** programming languages, which stay closer to an **ISA** (*Instruction Set Architecture*). An ISA is a specification which defines:
  - Supported instructions;
  - Data types;
  - Registers;
  - Addressing modes.

  In low-level languages, operations are smaller than those provided in high-level languages, so even simple algorithms usually take more instructions to express.

We can express this difference by taking the simple example of a loop:

1. In high-level languages such as *Python* and *C*, loops are expresed via a simple and predefined syntax which allows to compactly express iteration value initialization, update and the exit condition;
2. In a low-level language, all of these operations are distinct: the programmer has to explictly initialize an iteration value, perform a comparison to check for the exit condition, and manually update the iteration value.

Another idea to note is that high-level languages are usually designed to be *portable*, that is *interpreted* or *compiled* for a variety of systems. On the other hand, low-level

languages are usually designed for a single machine (a single type of processor), and porting to other platforms requires manual conversion work.

Programs written in high-level languages are usually compiled on the machines they are meant to run on: this means they are converted to low-level machine code which can be directly executed. For environments where a running a full compiler is not feasible (such as a development board), a technique known as *cross-compilation* can be used: the program is compiled ona second machine, into machine code for the first machine. ALl of this of course doesn't apply to low-level work, where we are expected to write the same routines from scratch, and likely differently, for each target architecture.

1.2.2 Data types and addressing

An ISA also defines the **data types** the CPU can address. For instance, the RISC architecture might allow a programmer to address bytes, half words (double bytes) and words (double half words). With this being said, we note that usually low-level languages interpret all data as *binary* data (complex types such as *integer*, *float*, ... are usually added in as high-level constructs).

There are two main methods of performing addressing:

- **Register-Memory** architectures: operands of instructions can be immediates (contained within the instruction encoding), or contained in memory *or* in registers. If *all* operands can be in registers or in memory (or in combinations), the architecture is called *register-plus-memory*. Clearly, as memory accesses are also operations that have to be carried out by the processor, these kinds of ISAs are more complex.
- **Register-Register** architectures: these are also called **load-store** architectures. Instructions of register-register architectures are explicitly divided into:
  - Memory access (*load* and *store* between memory and registers);
  - **ALU** (*Arithmetic and Logic Unit*) operations (only between registers). The ALU itself is one of the most important components of a processor.

  It follows that operands of ALU instructions are immediates or contained in registers.

In CISC (usually *register-to-memory*) instruction sets, the order of instructions can affect the number of memory accesses. The more the number of memory accesses increases, the more performance degrades. Load-store architectures, by their nature, do not suffer from this problem

1.2.3 CISC

CISC (*Complex Instruction Set Computer*) was the name given to the first computer designs. The main purpose is to complete a task in as few assembly lines as possible. This makes sense in an environment where programmers themselves are tasked with writing assembly code, as we said earlier that low-level languages tend to express the same concepts with more lines of code (we refer to this as a *semantic gap* between low and high-level languages).

Common examples of CISC CPUs are:

- x86 (1978) and x64 (2003) family, by Intel and AMD;
- Motorola 68000 series (1979);
- IBM System/360 family (1964);
- PDP-11 (1970) and VAX families (1977), by DEC.

Some characteristics of CISC are:

- Instruction encoding has *variable length*: for an immediate instruction we need to store the immediate, whereas for register to register or register to memory instructions we need to store register and memory addresses. As an example, x64 family instructions are between 1 and 6 bytes;
- The processor architecture has to be more *complex*, as a single instruction can express complex operations;
- The number of *clock cycles* required for execution of a single instruction is wildly variable: if an instruction requires a memory access, we need to wait for the memory to respond, which may happen on timescales far larger than those of the CPU;
- The resulting *code size* is far smaller than that of other architectures.

1.2.4 RISC

RISC (*Reduced Instruction Set Computer*) is the name given to a family of processors initially developed during the 1980s. The main purpose is to make the processor as simple as possible: this requires more work on the programmers' side, but allows the CPU to move much faster and execute many more instructions in the same amount of time as a CISC CPU. The limitation of more complex code, then, can be removed by making used of highly optimized *compiler* which take the toll of manual assembly programming away from programmers.

Common examples of RISC CPUs are:

- ARM family (1985), designed by ARM Holdings, which will also be the focus of the second part of this course;
- MIPS family (1984), by MIPS Technologies Inc.;
- RISC-V (2014), an example of a *register-to-register* ISA;
- SPARC family (1985), by Sun Microsystems;
- PowerPC (1991), by Apple, IBM, and Motorola.

Some characteristics of RISC are:

- The instructions themselves are *simpler*;
- The length of the instructions can be *fixed* (for example, in RISC-V they have a fixed size of 32 bits). A fixed instruction size sometimes requires some bits of the instruction encoding to remain unused: this is still preferred over variable length encoding as it allows the component of the CPU that performs the *fetch-decode* cycle to be far simpler;
- The processor architecture is *simpler*;
- The number of clock cycles required to execute an instruction is normally *lower* than in CISC, and there is less overall variance. This makes it easier to design and validate both RISC code and processors, as the amount of time taken by each instruction is easy to estimate (optimistically, 1 cycle per instruction).

  Common factors that disallow having exactly 1 cycle per instruction involve:

  – *Latencies* introduced by CPU optimizations, such as pipeline flushes and branch misses;
  – *Memory accesses*: load and store instructions always have to at least wait for the cache, and if a miss happens, for the memory, which we said is usually slower than the CPU;

  - Some *complex instructions* can be introduced back into the architecture: these give the programmer more power, but also require more clock cycles to complete.
- The resulting *code size* is larger than that of other architectures.

All of these characteristics contributed to making RISC architectures more popular than CISC architectures nowadays. In fact, some modern CISC processors actually convert the CISC instructions to an internal RISC representation, which then gets actually executed on the processor.

Also, the tradeoff given by longer code length is balanced by the fact that we are allowed to have *easier* CPUs, that is CPUs that are easier to *design* and *validate*. Some extensions have been introduced (such as the C "Compressed" extension for RISC-V) to make RISC-V instructions smaller and reduce the code memory footprint.

1.2.5 Comparison of addressing modes

As an example, let's compare the available addressing modes in x86 (a CISC architecture) and RISC-V (a RISC architecture). We then go over a specific detail we will encounter in ARM architectures.

- In **x86**, a number of addressing modes are available:
  - Register mode: `MOV AX, BX`
  - Immediate mode: `MOV AX, 2000`
  - Displacement or direct mode: `MOV AX, [DISP]`
  - Register indirect mode: `MOV AX, [DI]`
  - Based indexed mode: `MOV AX, [BX + DI]`
  - Indexed mode: `MOV AX, [SI + 2000]`
  - Based mode: `MOV AX, [BP + 0100]`
  - Based indexed displacement mode: `MOV AX, [SI + BP + 2000]`

  Such a large amount of modes could look a bit confusing. The idea is that modern x86 processors support **SIB** (*Scale-Index-Base*) addressing for arrays, *Mod R/M* bits, which allow for the following parameters:

  - **Base** is a general-purpose register, containing a base memory address;
  - **Index** is a general-purpose register contaning a memory offset;
  - **Scale** is a special constant, which can be 1, 2, 4, or 8;
  - **Displacement** is an arbitrary constant, offered as an immediate in the instruction.

  From these parameters, the processor computes an *Expected Address* (*EA*) is generally defined as:

$$EA = \text{Base} + \text{Index} \times \text{Scale} + \text{Displacement}$$

  Any of these components can be omitted, resulting in the complex array of addressing modes shown above (the indexed/based move differ mainly in semantics because of historical reasons).

  In the ISA, these addressing modes are specified by the instruction encoding, by the ModR/M byte, with an optional SIB byte. For more information, refer to the notes available at this very long URL: `https://raw.githubusercontent.com/seggiani-luca/appunti-rl/34228f66db395637bd1824d04f3130b977cc0ce4/master/master.pdf` (in italian).

We show one last table to summarize:

Addressing mode	Example	Loaded address
Register	`ADD AX, BX`	`AX`
Immediate	`ADD AX, FFh`	`FFh`
Displacement	`ADD AX, [FFh]`	mem[`FFh`]
Based	`ADD AX, [BX]`	mem[`BX`]
Indexed	`ADD AX, [CX]`	mem[`CX`]
Scale indexed	`ADD AX, [CX*4]`	mem[`CX` $\times$ 4]
Indexed based	`ADD AX, [BX + CX]`	mem[`BX` + `CX`]
Scale indexed based	`ADD AX, [BX + CX*4]`	mem[`BX` + `CX` $\times$ 4]

- In **RISC-V**, the same operations is significantly simpler. We have available:
  - ALU operations:
    * Register addressing mode: `ADD t0, t1, t2`
    * Immediate addressing mode: `ADDI t0, t1, 10`

    with the first register taken as the destination, in common RISC fashion.
  - Load / store operations, in base + displacement addressing mode: `LW t2, 4(t1)`, where the constant before the open parenthesis is an immediate offset, and the register contains a memory address. Of course, since for instant RV32x is a 32 bit architecture, 32 bit instructions cannot fully contain immediate addresses: this is taken into accoutn by special facilities in the processor which allow full address loading via multiple instructions.
- The same is available in ARM, with the distinction of having *pre-indexed* and *post-indexed* memory operations, that is control over the time when the base register is updated.

  Being more specific, ARM allows load and store instructions to optionally *alter* the address contained within a base register. 3 forms are available:
  - `ldr r0, [r1, #4]` is equivalent to immediate offset addressing in RISC-V, with `#4` being the immediate operand;
  - `ldr r0, [r1, #4]!`, with the additional `!`, means performing a *write-back* or **pre-indexing** operation. The calculated address, obtained by adding the offset, is *written back* into the base register before the load operation. The result we get is that of a load from the base address plus the offset, with the base address reflecting the change after execution. Compared to RISC-V, this saves an instruction;
  - `ldr r0, [r1], #4` means performing a **post-indexing** operation. This means increasing the value of the base register by offset, after performing the load. We can thing of this as being analogous to the `++` syntax available in C. This also saves an instruction compared to RISC-V.

  As we can see, the main point of this complication is to save a few instructions when accessing arrays, or generally where striding memory access patterns are needed in ARM architectures.

2 Lesson 23-09-26

2.1 Processor design

The first general-purpose computer was created in the late 50s. A Personal Computer, that can now be bought for about $500, is practically eguivalent (in terms of performance and memory) to what could be bought for about $1M in 1985.

This evolution has been made possible by:

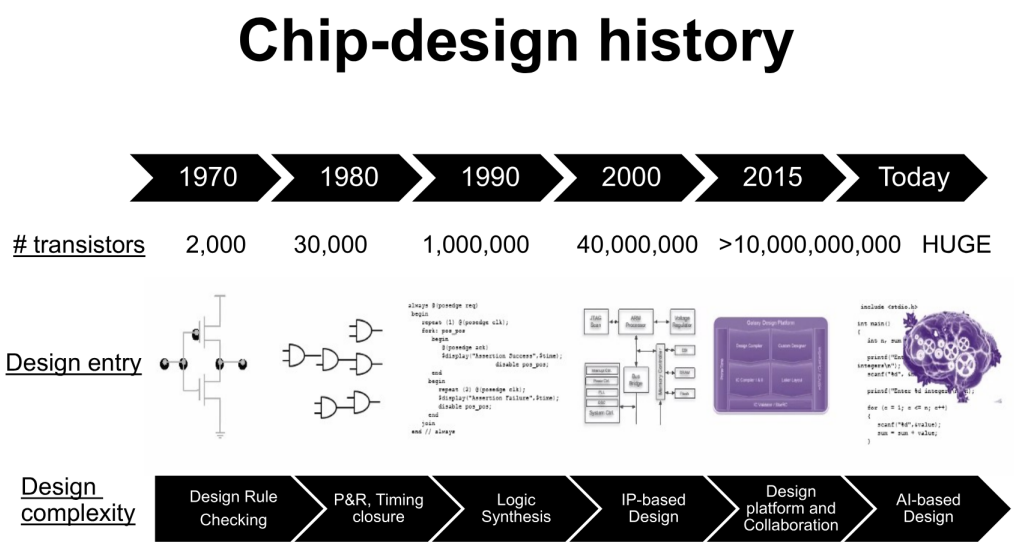
- Advances in semiconductor technology
- Innovations in computer architecture
- Advances in software.

Despite this the main architecture used today is still the same that was first applied to real computers, that is the *Von Neumann Architecture*.

2.1.1 Advanced architectures

Of course, there are modern advancements which have to be noted. We refer to these as *advanced architectures*:

- **Advances in computer architecture**: performance has been improved without fundamentally changing the Von Neumann model through techniques such as pipelining, which overlaps the execution of multiple instructions, and superscalar execution, which allows multiple instructions to be executed in parallel in a single clock cycle. Other major improvements include caches, branch prediction, and out-of-order execution.
- **Advances in software and tooling**: compilers have become increasingly sophisticated at optimizing programs for the underlying hardware, exploiting instruction-level parallelism, registers, caches, and specialized instructions. Better operating systems, programming languages, debuggers, profilers, and development tools have also made increasingly complex hardware practical to use.



2.1.2 Moore's law

Definition 2.1: Moore's law

The number of devices (i.e., transistors) that can be integrated into a single chip doubles every 18/24 months.

Despite the name, Moore's law describes an empirical relationship, not a scientific law. This type of observation, an experience curve effect, quantifies efficiency gains from learned experience in production. In fact, Moore's empirical evidence did not imply that the historical trend would continue; nevertheless, his prediction has held since 1975 and has since become known as a law.

We can also say that thanks to Moore's law, we are today able to integrate more and more transistors within one circuit, giving many powerful options to designers.

Many technologies are available to streamline chip design (for instance, *synthesis* and automated tools). Having more transistors within a single processor also allows for advanced features. For instance, something like a *floating point* processing unity executing alongside the main processor takes a large number of transistors to implement, which larger chip transistor densities allow.

2.1.3 Processor performance evolution

Processor **performance** can be measured by counting the number of operations executed or the time needed to execute them.

- User point of view: performance = response time (time between start and completion of an operation);
- System manager point of view: performance = *troughput* (total amount of work done in the time unit).

Processor performance evaluation is often performed by letting the computer to execute applications and observing its behavior. Unfortunately, the choice of the applications severely affects the performance. In the ideal case, one should use as workload the mix of applications the user will run. However, they are normally unknown, and largely variable from one user to another. Therefore, some **benchmarks** are selected to mimic real cases.

Processor performance has steadily increased since the 1970s, where the first mainframe machines started being ranked based on performance.

- From the 1970s to around the 1980s, yearly performance increases were of about $25\% \sim 30\%$. We have to recall that this was a time when mainframes and minicomputers dominated the market: performance improvements were driven mainly by advances in integrated-circuit technology, together with architectural improvements such as larger register sets, more buses, caches, and pipelining.
- In the 1980s, the introduction of **RISC** (*Reduced Instruction Set Computer*) architectures, as opposite to **CISC**, led to a yearly performance gain of around 50%. These allowed for major processor redesigns, which enabled optimization unfeasible (at least at the time) on CISC architectures;
- In the first years of the 2000s, processor performance growth hit a slow down because aiming for higher and higher clock speeds became unfeasible. There were 3 main reasons:

  - **Power** issues: higher clock speeds meant higher power dissipation, which cooling could only partially keep under control:
  - Lower **ILP** (*Instruction Level Parallelism*): this was disadvantageous for the *superscalar* architectures used at the time, characterized by high pipeline depth and instruction level parallelism;
  - Unchanged **memory latency** remained a big bottleneck.

- The solution most vendors adopted from the mid 2000s onwards, and which to this day is widely adopted, is *multi-core* processors. In fact, since 2004, major industries stopped the race for higher-performance single processor projects running at higher clock frequencies, and embraced *multi-core* CPUs.

  In a **multi-core** processor, we effectively have multiple processors (or processor *cores*) running in parallel on a single, or multiple tasks. Note that multi-core setups increase *throughput*, and not *response time*.
- In more recent times (mid 2010s) we have seen a 12% year improvement, or doubling every 8 years (mainly due to the limits on instruction parallelism). This has slowed down to a 3.5% performance improvement, or doubling every 20 years, in the last 10 years (at the time of writing).

Today, we can not only talk about processor improvements, but the fact that modern computer systems employ various types of *accelerators*, that is circuits intended to support specific applications alongside the CPU, like graphics or AI (an example is given by **NPU**s, *Neural Processing (Units)*).

In the recent years, the widespread adoption of Artificial Intelligence (AI) applications prompted the introduction of new computing architectures denoted as AI accelerators, such as **GPU**s (*Graphics Processing (Units)*). GPUS were originally introduced for graphics applications, but their current success is due to their ability to effectively manage large and regular data structures. The GPU market (in 2024 corresponding to about 18 B$), is expected to grow to 406 B$ by 2033 (with a Compound Average Grow Rate, or *CAGR*, of 41,39%). The CAGR of the CPU market is about 7%.

2.1.4 Costs

In processor, and more generally computer design, **costs** have to be taken into account. These can be divided into two main categories:

- **Design costs**: these are largely *fixed* or *non-recurring* costs, usually referred to as **NRE**s (*Non-Recurring Engineering costs*). They include activities such as architecture design, circuit design, verification, physical design, software development, and mask production. NRE costs grow rapidly with chip complexity and with the sophistication of the manufacturing process. In particular, moving to a newer process node requires expensive new design tools, masks, **IP** (*Intellectual Property*), and extensive verification.

  Since these costs are paid before mass production, they have to be amortized over the number of chips sold. Therefore, a high-volume product can tolerate large NRE costs because they are distributed over many units, whereas for a low-volume product they can represent a significant portion of the final cost;
- **Manufacturing costs**: these are the *recurring* costs associated with producing each chip. They include wafer fabrication, packaging, assembly, and testing. The semiconductor manufacturing process itself requires extremely expensive fabrication facilities (*fabs*), whose cost increases substantially for advanced process nodes.

An important factor when evaluating manufacturing costs is the **yield**, i.e., the percentage of manufactured chips that satisfy all specifications and pass the testing phase. Not every die on a wafer is functional: manufacturing defects can make some chips unusable. Consequently, a low yield increases the effective cost of every working chip.

Yield normally improves over time as the manufacturing process matures. This phenomenon is known as the **learning curve**: defects are progressively reduced, manufacturing parameters are optimized, and the production process becomes more reliable. As yield increases, the effective cost per functional chip decreases.

Testing and validation also contribute significantly to the recurring cost of a processor. Complex processors require extensive electrical testing and, in many cases, additional screening and characterization before they can be sold. Thus, manufacturing cost is not simply the cost of fabricating the silicon itself, but includes all the steps required to obtain a tested and sellable device.

2.1.5 The computer market

The computer *market* is currently split in 5 different areas:

- *Personal Mobile Devices* (**PMD**s): this area includes smartphones and tablet computers. Emphasis is on energy efficiency and real-time applications. Here, system price is around $100 - $1,000, with the microprocessor price at $10 - $100. Architectures are usually highly parallel (and in fact smartphones were the earliest devices to make heavy use of multi-core processors);
- Desktop computing: this area covers from PCs to workstations (and in some sense, even laptops). The main target of this area is to optimize the price-performance ratio. The system price is around $300 - $2,500, and of course the microprocessor price follows $50 - $500. Power is much less important than before, as we don't use batteries but power off the grid;
- Servers: these systems provide larger-scale and more reliable computing services. The main parameters of products in this area are:
  - Availability;
  - Scalability;
  - Throughput.

  System price is around $5,000 - $10,000,000, with microprocessor price around $200 - $2,000;
- Clusters / *Warehouse Scale Computers* (**WSC**): systems that offer "Software as a Service (*Saas*)" or "Platform as a Service (*PaaS*)". The emphasis is on availability, price-performance, and power consumption. *Supercomputers* make up a part of this category, where the emphasis is on floating-point performance and fast internal networks. The system price is high at around $100,000 - $200,000,000, with the microprocessor price around $50 - $250 (because at times we might use off the shelf processors in massively parallel setups);
- Embedded computers: this area is a major application area of the computer market. It covers all special-purpose computer-based applications (from microwaves to coffee machines, from automotive to videogames). Adopted microprocessors vary from cheap low-end 8-bit processors to very efficient (and expensive) high-end processors. The system price ranges from $10 - $100,000, with microprocessor price at around $0.01 - $100 (possibly extremely cheap).

Special reguirements often existing in this area are:

- Real-time performance reduirements;
- Memory minimization;
- Power consumption minimization;
- Reliability constraints.

Embedded problems are often solved resorting to one of the following "standard" solutions:

- Standard processor, custom logic, custom software. We call the combination of standard processors with custom logic **ASIC** (*Application Specific Integrated Circuit*);
- Standard processor, custom sofware;
- Standard DSP, custom software.

Programmable devices (such as **FPGA**s) are playing a growing role.

2.1.6 Power

Continuous increase in system complexity and device integration often leads to problems with **power consumption.** Power consumption may be critical under two aspects:

- **Power**: *static* power, consumed when a device is just running without doing work, and *dynamic* power, which mainly depends on the frequency devices run at (and also the square power). Until now it has been dominated by dynamic power, i.e., that consumed by each transistor when switching between different states.
  - The **dynamic** power for each transistor can be evaluated by the usual formula:

$$P_{dynamic} = \frac{1}{2} f V^2 C_{eq}$$

  - Whereas **static** power can be evaluated by:

$$P_{static} = VI$$

    and amounts aroudn 50% of power consumption.

  For this very reason, voltage continuously dropped in the last years.
- **Energy** consumed: it is given by:

$$E = C_{eq} V^2$$

  It is mainly of interest for *portable* devices, where we are draining a battery (these include mobile phones and laptops, like the one the author of this notes is writing on, which will likely shut down if a power outlet is not found stat).

2.1.7 Depedendability

Dependability is the quality of the system to deliver a correct service. Dependability of computer systems is traditionally very high, but it can be lowered by:

- Bugs in the design of the hardware;
- Bugs in the software;
- Defects in the hardware (introduced by the manufacturing process);

- Faults happening during the product operation.

Many **safety critical** systems exist. In the past, possible misbehaviors of computer systems were critical in some areas, such as

- Space;
- Avionics;
- Nuclear plants control.

In more recent years, computer-based systems expanded to other safety-critical areas, such as:

- Rail-road traffic control
- Automotive
- Biomedical
- Telecommunications.

Metrics for dependability are defined as:

- *Mean Time To Failure* (**MTTF**) or *Failures In Time* (**FIT**), which is the reciprocal. 1 FIT equals 1 failure in one billion hours;
- *Mean Time To Repair* (**MTTR**);
- *Mean Time Between Failures* (**MTBF**).

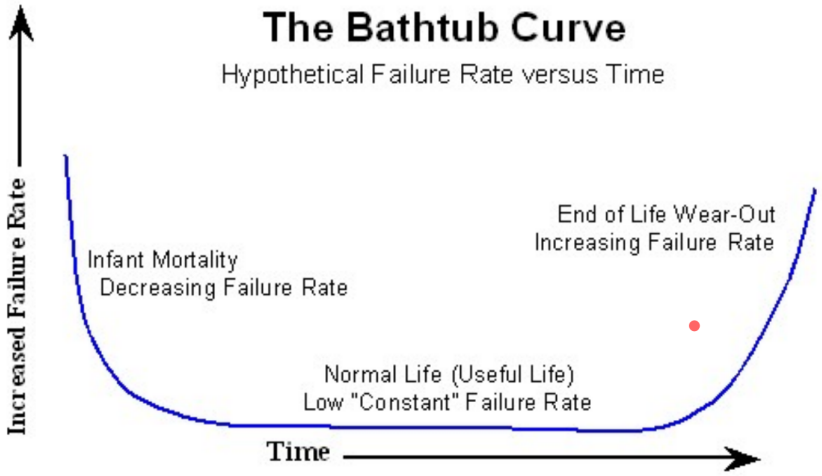
The three measures are related by the formula:

$$MTBF = MTTF + MTTR$$

Finally, availability is the probability that a system works correctly in a generic time instant (usually measured in FITs).

The **bathub curve** describes the probabilty of faults given the age of a component. We give it as a final example of how to take dependability into account.

- The *infant mortality* zone is where faults of components show themselves early. Controlled *burn-in* can partially solve this problem;
- The *normal life* zone is where expect to spend time using devices;
- The *end of life* zone is where failures hidden in components which otherwise worked fine start appearing. This is of particular interest to data centers where components are heavily stressed.



2.1.8 Reproducibility

Going back to the topic of *performance*, we introduce **reproducibility**. Information about execution times on benchmarks should allow reproducibility. This means reporting detailed information about:

- Hardware (system configuration);
- Software (OS, compiler, program);
- Program input.

A common issue is comparing and summarizing performance:

- **Problem 1**: I know the performance of one machine on a set of programs: which is its global performance?
- **Problem 2**: I know the performance of two machines on the same set of programs: which is their relative performance?

A number of metrics have been proposed, such as total execution time, arithmetic mean and weighted means.

2.1.9 Ahmdal's law

Definition 2.2: Ahmdal's law

We define the *speedup* from a given architecture enhancement as:

$$\text{speedup} = \frac{\text{performance with enhancement}}{\text{performance without enhancement}}$$

The speedup resulting from an enhancement depends on two factors:

- $\text{fraction}_{\text{enhanced}}$: the fraction of the computation time that takes advantage of the enhancement;
- $\text{speedup}_{\text{enhanced}}$: the size of the enhancement on the parts it affects.

We can compute the execution time after the speedup as:

$$\text{execution time}_{\text{new}} = \text{execution tiem}_{\text{old}} \cdot ()$$

Applying the definition of speedup, we get:

$$\text{speedup} = \frac{\text{performance with enhancement}}{\text{performance without enhancement}} = \frac{1}{(1 - \text{fraction}_{\text{enhanced}}) + \frac{\text{fraction}_{\text{enhanced}}}{\text{speedup}_{\text{enhanced}}}}$$

Ahmdal's law is particularly useful in the study of parallel architectures, as it expresses the speedup given by optimizing parallelizable parts of computing tasks. More specifically, we use it to explain why we can never have $S = N$ with N processor cores.

We can say that there will always be intervals of sequential execution (T_{seq}), spent executing instructions that cannot be parallelized, such as I/O operations, conditional statements, inherently sequential algorithms, etc... Then the speedup S can be defined as:

$$S = \frac{T_1}{T_{\text{seq}} + \frac{T_1 - T_{\text{seq}}}{N}}$$

with T_1 being the full execution time, i.e., the speed-up gain comes only from the parallelizable parts of the program ($T_1 - T_{\text{seq}}$), while the sequential parts (T_{seq}) cannot benefit significantly from parallelism.

The problem is that, taking the limit for $N \to \infty$, we obtain:

$$\lim_{N\to\infty} S = \lim_{N\to\infty} \frac{T_1}{T_{\text{seq}} + \frac{T_1 - T_{\text{seq}}}{N}} = \frac{T_1}{T_{\text{seq}}}$$

Thus, as N increases, the speed-up becomes dominated by the sequential parts: the parallelizable portions are progressively reduced, while the sequential portions remain and ultimately become the limiting factor for the overall system performance.

2.1.10 The CPU performance equation

Let's talk about measuring the time required to execute a program There are several possible approaches:

- By observing the real system;
- By simulation;
- By applying the **CPU performance equation**.

Definition 2.3: CPU performance equation

CPU execution time is given by:

$$\text{CPU Time} = \left(\sum_i \text{CPI}_i \times \text{IC}_i \right) \times T_C$$

where:

- CPI_i is the number of clock cycles required by instruction i;
- IC_i is the number of times instruction i is executed in the program;
- T_C is the clock period.

This equation is quite basic, as it doesn't take any processor optimizations into account. Still, it is a feasible metric for processor performance estimation, before the processor is even actually implemented.

2.2 Instruction Set Principles

Every processor owns an *Instruction Set Architecture* (**ISA**). The ISA provides the information about the processor reduired by programmers and compilers to produce correctly working code:

- The set of available *instructions*;
- Instruction *encodings*;
- Available *registers*;
- etc...

Different processors may share the same ISA (e.g., Intel and AMD processors, or processors of the same family).

The Instruction Set Architecture (ISA) is basically how the computer is seen by the programmer or the compiler. There are many alternatives possible for the ISA designer,

any choice modifies the way to encode the instructions. The different alternatives may be evaluated in terms of:

- Processor performance;
- Processor complexity;
- Compiler complexity;
- Code size;
- Power consumption;
- etc...

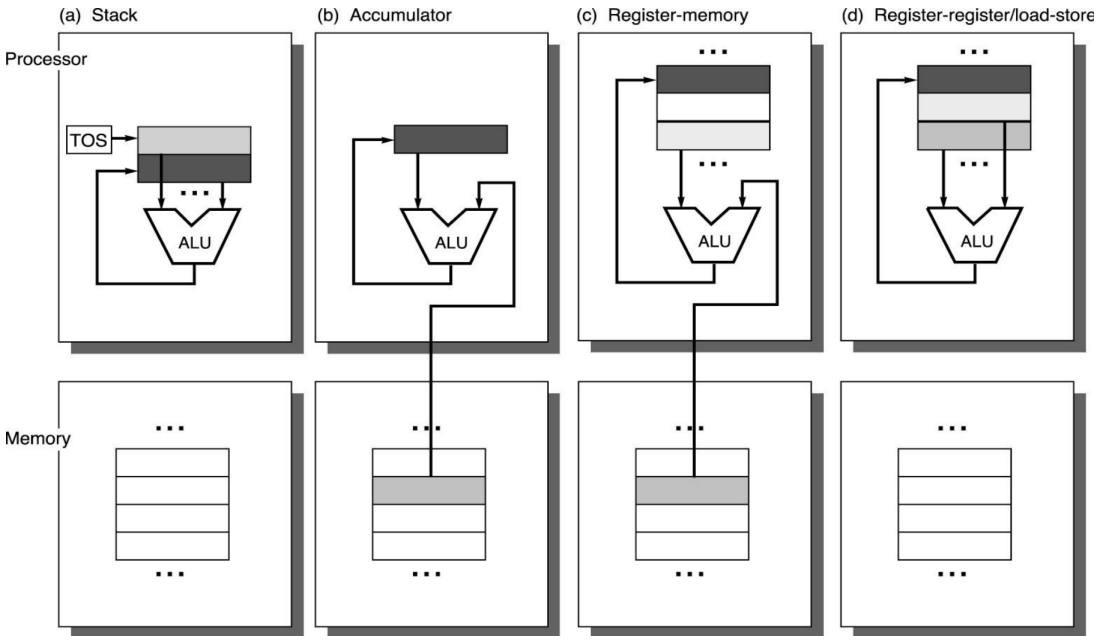
Different product areas may assign different weights to the above parameters.

An ISA specification is particulary important because it is usually the only set of information released by a company about a processor. Of course, the internal implementation details are trade secrets, but the ISA specification is made available in order to allow programmers to develop for a certain platform.

2.2.1 CPU taxonomies

CPUs (and their ISAs) are often classified according to the type of their internal storage:

- Stack;
- Accumulator;
- Registers:
  - Register-memory, as in x86;
  - Register-register (load-store). Basically all RISC architectures are of this kind;
  - Memory-memory (no real cases).



Currently, all processors are *General-Purpose Register* (**GPR**) machines. The reason for this is:

- Registers are faster than memory;
- Registers are easier for a compiler to use (or better, to *optimize*).

CPUs can also be classified according to:

- Typical number of operands per ALU instruction (2 or 3). In RISC-V and ARM, this is almost always 2;
- Typical number of memory operands per ALU instruction (from 1 to 3, for architectures that feature memory operands in the first place in instructions rather than load/store).

2.2.2 Memory addressing

We have some orthogonal characteristics relating to memory *addressing*:

- **Little Endian** vs. **Big Endian**: processors usually address memory in bytes, organized into words. The order of the bytes within a word matters:
  - *Little Endian* puts the byte with the lowest address (X...X000) at the least significant position. The address of the data is that of the least significant byte;
  - *Big Endian* puts the byte with the lowest address (X...X000) at the most significant position. The address of the data is that of the most significant byte.
- **Aligned** vs. **misaligned** accesses:
  - *Aligned* means allowing only aligned accesses not straddling word (usually 2 byte) bounds. It is a limitation;
  - *Disaligned* access, on the other hand, requires: hardware overhead and performance overhead.

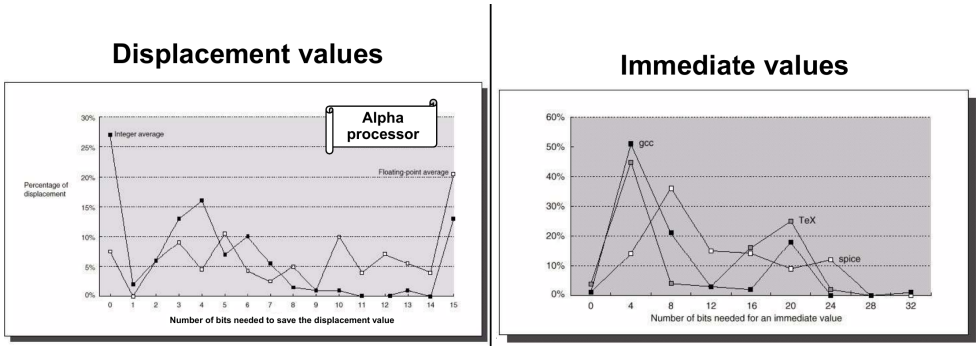
We shall also note that in GPR machines an addressing mode specifies a constant, a register, or a memory location (through its effective address).

We remember the addressing modes noted in section 1.2.5 for x86. Some of these are shared with ARM and RISC-V RISC architectures (mainly register mode, with destination register, immediate and displacement modes for load/store operations).

By carefully choosing the addressing modes, one can obtain some important consequences:

- Reducing the number of instructions;
- Increasing the CPU architecture complexity;
- Increasing the average *Cycles Per Instruction* (**CPI**).

An interesting thing to note is the size devoted to immediate operands in immediate instructions. That is, how large can the immediate value be, when it is embedded in the addressing part of the instruction? Note that memory displacements basically are a special case of immediate operands, and as such fall in the same category and have the same problem. In fact, we had already noted this issue in section 1.2.5.



An experimental evaluation can be performed. From the plots, compiled from a DEC alpha processor for displacement and for typical compiled applications for immediates, we get that:

- Displacement, immediate and register indirect modes represent from 75% to 99% of the addressing modes;
- The address size for displacement mode should be from 12 to 16 bits (75% to 99% of the displacements);
- The size of the immediate field should be at least 8 or 16 bits (50% and 80% of the cases, respectively).

RISC-V uses the solution of doing 12 bits immediates. As we previously stated, bigger immediates have to be transferred to register in multiple instructions. This tradeoff is made valid by the simplicity gains of the processor architecture.

2.3 Operations in the instruction set

Clearly, a key component of an ISA is the **instruction set** made available to the programmer. Different categories of instructions exist:

Operator type	Examples
Arithmetic and logical	Integer arithmetic and logical operations: add, and, subtract, or
Data transfer	Loads-stores (move instructions on machines with memory addressing)
Control	Branch, jump, procedure call and return, traps
System	Operating system call, virtual memory management instructions
Floating point	Floating-point operations: add, multiply
Decimal	Decimal add, decimal multiply, decimal-to-character conversions
String	String move, string compare, string search
Graphics	Pixel operations, compression/decompression operations

We should be making the common case fast: not all the instructions are executed with the same frequency! When designing and implementing an instruction set, the most commonly executed instructions should be made faster than others.

3 Lesson 25-09-26

Let's continue talking about instruction set architectures. We recall that this part is a preliminary step in the design of every processor. The instruction set itself is, we said, the minimum set of information that the manufacturing company provides to users (programmers) of its processors. All the rest of the design remains confidential to the processor company (logical and physical architecture, ...).

Control flow instructions determine the control flow in the program. They can be distinguished in four categories:

- **Conditional branches**: this involves a comparison of values, which if evaluated to a true value, triggers a jump to some other location in code. Otherwise nothing happens;

- **jumps**: these are jumps to arbitrary code which always happen;
- Procedure **calls**: these involve saving some state about the CPU before jumping;
- Procedure **returns**: these involve recovering the state saved by procedure calls.

Conditional branches are by far the most frequently executed control flow instructions.

Control flow instructions affect the value of the *program counter* register, which points to the current instruction. The destination address It is normally specified as a displacement with respect to the current value of the program counter. In this way:

- We save bits, since the target instruction is often close to the source one;
- The code is position-independent (**PIC**, *Position Indipendent Code*).

We should make a note about **register** based indirect jumps and procedure calls. They allow:

- To write code including jumps whose target is not known at compile time;
- To implement case or switch statements
- To support dynamically shared libraries (i.e., libraries which are loaded only when called)
- to support virtual functions (i.e., calling different functions depending on the data type).

We not show a table which compares jumps in multiple architectures.

Name	Examples	Test	Advantages	Disadvantages
Condition code (CC)	80x86, ARM, PowerPC, SPARC, SuperH	Tests special bits set by ALU operations, possibly under program control	Sometimes condition is set for free.	CC is extra state. Condition codes constrain the ordering of instructions because they pass information from one instruction to a branch
Condition register/limited comparison	Alpha, MIPS	Tests arbitrary register with the result of a simple comparison (equality or zero tests)	Simple	Limited compare may affect critical path or require extra comparison for general condition
Compare and branch	PA-RISC, VAX, RISC-V	Compare is part of the branch. Fairly general compares are allowed (greater than, less than)	One instruction rather than two for a branch	May set critical path for branch instructions

Let's now focus on **procedures**. When calling a procedure, as we said, some information need to be saved:

- The **return address**: this is the address of the instruction *right after* the call instruction. *Where* this address is stored depends on the architecture:

  - x86 performs *stack* saving, accessing memory at the address pointed to by the stack pointer and decrementing it. This is referred to as *pushing* the address: *popping* the address involves incrementing the stack pointer and recovering the address;
  - Architectures like RISC-V implement **JAL** (*Jump And Link*), where the return address is stored in a register. This saves one memory access for leaf procedure calls (that is procedures which don't call other procedures). If more calls are needed stack or other register saving can be performed manually.
- The accessed registers, based on whether our **ABI** (*Application Binary Interface*) defines *caller saving* or *callee saving* for certain registers (both can be defined, on different registers).
  - **Caller saved** registers are meant to be saved before the procedure call, if the caller intends to keep their value;
  - **Callee saved** registers are meant to remain unchanged after a procedure call. The called procedure can implement this by saving registers itself (through other registers or stack pushing), or by not using them at all.

To summarize instruction sets, we can say that:

- Few instructions are responsible for most of the execution time: load, store, add, subtract, move, register-register, and, shift, compare equal, compare not equal, branch, jump, call, and return.
- PC-relative branch displacements of at least 8 bits are the best choice.
- Register-indirect and PC-relative addressing can also be used in procedure call and return

3.0.1 Instruction set encodings

Instructions have to be encoded in a format the processor can *fetch* and *decoded* before executing. Instruction set **encoding** depends on:

- Which instructions compose the *instruction set*;
- Which *addressing modes* are supported (we have talked about register and memory addressing modes in section 2.2.2).

When a high number of addressing modes is supported, an address specifier field is used to specify the addressing mode and the registers which are possibly involved (this is the CISC philosphy). When the number of addressing modes is low, they can be encoded together with the opcode (this is the RISC philosophy).

We have 3 main types of instruction encoding:

- **Variable** encoding, where instructions have no fixed size and can grow based on immediates, displacements, ... This is the approach used by x86 and VAX architectures.

  Other characteristics are:

  - Supports any number of operands;
  - Instructions have a variable length;
  - Lower performance;
  - Minimum code size.
- **Fixed** encoding, where instructions are fixed in size and have fields which usually mean the same thing across instructions. This is the approach used by many RISC architectures, such as RISC-V, ARM, MIPS, PowerPC and SPARC.

Other characteristics are:

- Fixed number of operands;
- Address specifier included in the opcode;
- Fixed instruction length;
- Maximum performance;
- Larger code size.

- **Hybrid** encoding, where instructions are mostly fixed, but shorter instructions (often called *compressed* instructions) might appear for memory saving. This solves the problem of many RISC architectures, where the additional instructions that have to be included to perform operations grow code size (we talked about this in 1.2.4). This is the approach used by many compressed extensions for RISC architectures, such as RISC-V compressed (RV32IC), ARM Thumb2, microMIPS, and even older architectures like IBM 360/370.

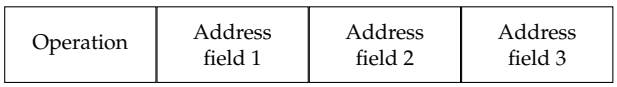
  Other characteristics are:

  - Multiple formats (specified by the opcode);
  - Allow trading-off between code size and performance.

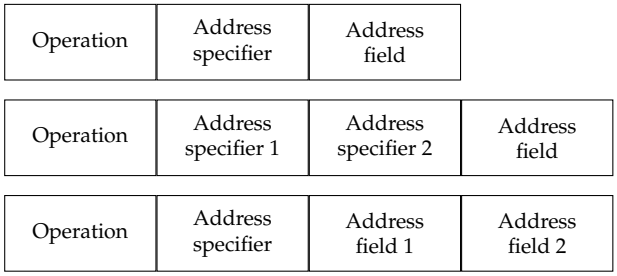
  We show a summarizing figure:



(A) Variable (e.g., Intel 80x86, VAX)



(B) Fixed (e.g., RISC-V, ARM, MIPS, PowerPC, SPARC)



(C) Hybrid (e.g., RISC-V Compressed (RV32IC), IBM 360/370, microMIPS, Arm Thumb2)

We note that, when designing encodings, the designer should address several conflicting issues:

- The code size;
- The size of the Instruction Set, the number of addressing modes, and the number of registers;
- The complexity of the fetch and decoding hardware.

We should also note that nowadays, ssembly-level programs are now produced by compilers, only. This eases the pressure off programmers, but means that the CPU desi-

gner and the compiler writer must interact and cooperate.

For instance, choosing which variables have to be put in which registers and when, is one of the crucial optimization phases in a compiler. This problem is based on graph coloring and can be solved easily if the number of registers is high (> 16). Of course, maximum performance would be achieved with an infinite number of registers, but this cannot be achieved as silicon is limited.

Optimizing variable access time by allocating variables to registers is only possible for those stored in the stack or for global variables in memory. It is impossible for variables belonging to the heap, due to the aliasing problem (i.e., the access to a variable is done through pointers).

To summarize, the CPU architect can help the compiler developer by:

- Making the frequent case fast and the rare case correct;
- Regularity;
- Providing primitives, not solutions;
- Simplifying trade-offs among alternatives;
- Providing instructions that bind the quantities known at compile time as constants.

The baseline recommendations are those of:

- **Orthogonality**, that is designing instructions that do not care about the *registers* which are used, or other specific constraints. Many famous instruction sets such as x86 are *not* orthogonal: for instance, multiplication and division instructions only operate on a specific subset of registers, in specific roles;
- **Simplcity**,
- Having at least 16 registers;

3.1 RISC-V introduction

We now give an introduction to the **RISC-V**. This and *ARM* will be the two architectures we cover during the course.

RISC-V is an open standard **ISA** (*Instruction Set Architecture*), having a few instructions in 3 different versions: 32, 64 and 128 bits. The fact that the ISA is fully open means that it is:

- Extremely easy and cheap to implement;
- Customizable and extensible.

The RISC-V ISA itself is defined as a base ISA (RV32I) plus extensions and user customizations. The base extensions feature things such as multiplication, protection, floating point operations, ...

Today, RISC-V is:

- Widely thought in universities;
- Increasingly adopted by companies (e.g., in the automotive domain);
- Strongly supported by the European Commission.

3.1.1 Technical characteristics

Let's get technical. Some generalities about RISC-V are:

- Freely licensed open standard;
- **Load/store** architecture;

- Fixed **32-bit** instruction set (whether RV32I, RV64I or RV128I is implemented);
- Additional **ISA extensions**, for example for security, floating point, and vector operations;
- The supported **addressing modes** are:
  - Displacement;
  - Immediate;
  - Register indirect.

The supported **data types** are:

- Byte (8 bits);
- Half Words (16 bits);
- Words (32 bits);
- Double Words (64 bits);
- 32-bit single precision floating-point;
- 64-bit double precision floating-point.

In regards to **encoding**, RISC-V allows a **variable** encoding in multiples of 16-bit blocks if code size reduction or specific implementations are needed.

Fixed-length 32-bit instructions are used in the base versions:

- 32-bit instructions have their lowest two bits set to 11;
- 16-bit instructions may have their lowest two bits equal to 00, 01, or 10.

In the case of 32-bit instructions, code memory alignment is a constraint; in other cases, the alignment is relaxed.

A little-endian memory system is the default choice.

3.1.2 Extensions

It counts with three base instructions sets plus a variety of optional extensions. For example, a 64-bit RISC-V possible ISA is defined as RV64IMAFD, known as RV64G, general-purpose scalar instruction set. The 32-bit version is called RV32G.

A table of common extensions is shown below.

Name of base or extension	Functionality
RV32I	Base 32-bit integer instruction set with 32 registers
RV32E	Base 32-bit instruction set but with only 16 registers; intended for very low-end embedded applications
RV64I	Base 64-bit instruction set; all registers are 64-bits, and instructions to move 64-bit from/to the registers (LD and SD) are added
M	Adds integer multiply and divide instructions
A	Adds atomic instructions needed for concurrent processing; see Chapter 5
F	Adds single precision (32-bit) IEEE floating point, includes 32 32-bit floating point registers, instructions to load and store those registers and operate on them
D	Extends floating point to double precision, 64-bit, making the registers 64-bits, adding instructions to load, store, and operate on the registers
Q	Further extends floating point to add support for quad precision, adding 128-bit operations
L	Adds support for 64- and 128-bit decimal floating point for the IEEE standard
C	Defines a compressed version of the instruction set intended for small-memory-sized embedded applications. Defines 16-bit versions of common RV32I instructions
V	A future extension to support vector operations (see Chapter 4)
B	A future extension to support operations on bit fields
T	A future extension to support transactional memory
P	An extension to support packed SIMD instructions: see Chapter 4
RV128I	A future base instruction set providing a 128-bit address space

3.1.3 Registers

The RV64G register bank consists of a register bank (or register file) of 32 elements, that is 32 integer registers `x0`, `x1`, ..., `x31`. The same amount of registers is available for floating point single (32-bit) and/or double precision (64-bit).

Register `x0` (called register *zero*) always returns the value 0, and cannot be written to. One reason for having such a register is shown below (3.1.4).

Other registers have roles defined by the ABI (though when doing bare metal we choose to respect them or not). We show a figure below of these roles.

Register	ABI Name	Description	Register	ABI Name	Description
x0	zero	hardwired zero	x16	a6	function argument 6
x1	ra	return address	x17	a7	function argument 7
x2	sp	stack pointer	x18	s2	saved register 2
x3	gp	global pointer	x19	s3	saved register 3
x4	tp	thread pointer	x20	s4	saved register 4
x5	t0	temporary register 0	x21	s5	saved register 5
x6	t1	temporary register 1	x22	s6	saved register 6
x7	t2	temporary register 2	x23	s7	saved register 7
x8	s0 / fp	saved register 0 / frame pointer	x24	s8	saved register 8
x9	s1	saved register 1	x25	s9	saved register 9
x10	a0	function argument 0 / return value 0	x26	s10	saved register 10
x11	a1	function argument 1 / return value 1	x27	s11	saved register 11
x12	a2	function argument 2	x28	t3	temporary register 3
x13	a3	function argument 3	x29	t4	temporary register 4
x14	a4	function argument 4	x30	t5	temporary register 5
x15	a5	function argument 5	x31	t6	temporary register 6

3.1.4 Addressing modes

Addressing modes for data transfers are *immediate* and *displacement*, both counting with an encoded field of 12 bits.

- **Immediate** addressing is done through the usual syntax, remembering that we have 3 registers per instruction (destination, source 1 and source 2, as per RISC ABIs): `addi x1, x2, 32;`
- **Displacement** is specified on indirect register addressing in load/store instructions: `lw x1, 32(x2)`. Let us also note that the `w` in `lw` stands for "word", as in load/store instructions in x86 fashion need their operand size specified in the opcode mnemonic. Of course a displacement of 0 equals to register indirect addressing. An interesting thing is how absolute displacements are specified: we have to include a register for all load instructions, and we can use just the 12 bit displacement as an address if we make that address `x0` (which we know always evaluates to 0).

3.1.5 Instruction formats

Instruction formats are an integral part of the RISC-V ABI. The *green card* available at `https://www.cs.sfu.ca/~ashriram/Courses/CS295/assets/notebooks/RISCV/RISCV_CARD.pdf` remains an extremely useful resource.

31 27	26 25	24 20	19 15	14 12	11 7	6 0	
funct7		rs2	rs1	funct3	rd	opcode	R-type
imm[11:0]			rs1	funct3	rd	opcode	I-type
imm[11:5]		rs2	rs1	funct3	imm[4:0]	opcode	S-type
imm[12|10:5]		rs2	rs1	funct3	imm[4:1|11]	opcode	B-type
imm[31:12]					rd	opcode	U-type
imm[20|10:1|11|19:12]					rd	opcode	J-type

Each instruction belongs to one of 5 formats, which specifies the semantics of fields within the instruction encoding. We see that fields like the instruction *opcode*, and the `funct3` and `funct7` values are shared between most formats.

- **R-type**: these are *register-to-register* instructions.

Field	Decription
`opcode`	7 bit primary operation opcode
`rd`	5 bit specifier for destination register
`funct3`	Specifies the type of operation (`add` / `and` / ...)
`rs1`	5 bit specifier for first register
`rs2`	5 bit specifier for second register
`funct7`	Selects a specific suboperation from a main operation (for instance sign extending and non-sign extending shifts)

- **I-type**: these are instructions with *immediates*.

Field	Decription
`opcode`	7 bit primary operation opcode
`rd`	5 bit specifier for destination register
`funct3`	Specifies the type of operation (`addi` / `andi` / ...)
`rs1`	5 bit specifier for first register
`immediate`	12 bit signed immediate used for logical operands, arithmetic signed operands, load/store address byte offsets, and PC-relative branch signed instruction displacement. Note that this is *sign extended*, as all other immediates in RISC-V! (for simplicity).

- **S-type**: these are *store* instructions.

Field	Decription
`opcode`	7 bit primary operation opcode
`rs1`	5 bit specifier for register to add to the 12 bits immediate
`rs2`	5 bit specifier for register to copy in memory
`funct3`	Specifies the size of the operation
`imm` fields	These two values compose the 12-bit signed value.

- **B-type**: these are *branch* (basically conditional additions affecting the program counter) instructions.

Field	Decription
`opcode`	7 bit primary operation opcode
`rs1`	5 bit specifier for first register to compare
`rs2`	5 bit specifier for second register to compare
`funct3`	Specifies the type of comparison
`imm` fields	These two values compose the 12 bit signed value.

- **U-type**: these are so-called *upper immediate* instructions, used to target the upper 20 bits 12 bit immediate instructions can't reach.

Field	Decription
`opcode`	7 bit primary operation opcode
`rd`	5 bit specifier for destination register (usually the program counter)
`imm` fields	These two values compose the 20 bit signed value.

- **J-type**: these are *jump* instructions.

Field	Decription
`opcode`	7 bit primary operation opcode
`rd`	5 bit specifier for destination register (usually the program counter)
`imm` fields	These two values compose the 20 bit signed value.

3.1.6 Available instructions

The instructions available on the RV32G, grouped by function:

- Load and store
- ALU operations
- Branches and Jumps
- Floating Point
- Miscellaneous

In the following some of the most used instructions and pseudo-instructions are described.

Load/store: as we know, RISC-V processors use a load-store architecture. The main memory is accessed only through load and store instructions.

Example instruction	Instruction name	Meaning
`ld x1,80(x2)`	Load doubleword	Regs[x1] ← Mem[80+Regs[x2]]
`lw x1,60(x2)`	Load word	Regs[x1] $\leftarrow_{64}$ Mem[60+Regs[x2]]$_0$)^{32} ## Mem[60+Regs[x2]]
`lwu x1,60(x2)`	Load word unsigned	Regs[x1] $\leftarrow_{64}$ 0^{32} ## Mem[60+Regs[x2]]
`lb x1,40(x3)`	Load byte	Regs[x1] $\leftarrow_{64}$ (Mem[40+Regs[x3]]$_0$)^{56} ## Mem[40+Regs[x3]]
`lbu x1,40(x3)`	Load byte unsigned	Regs[x1] $\leftarrow_{64}$ 0^{56} ## Mem[40+Regs[x3]]
`lh x1,40(x3)`	Load half word	Regs[x1] $\leftarrow_{64}$ (Mem[40+Regs[x3]]$_0$)^{48} ## Mem[40+Regs[x3]]
`flw f0,50(x3)`	Load FP single	Regs[f0] $\leftarrow_{64}$ Mem[50+Regs[x3]] ## 0^{32}
`fld f0,50(x2)`	Load FP double	Regs[f0] $\leftarrow_{64}$ Mem[50+Regs[x2]]
`sd x2,400(x3)`	Store double	Mem[400+Regs[x3]] $\leftarrow_{64}$ Regs[x2]
`sw x3,500(x4)`	Store word	Mem[500+Regs[x4]] $\leftarrow_{32}$ Regs[x3]$_{32..63}$
`fsw f0,40(x3)`	Store FP single	Mem[40+Regs[x3]] $\leftarrow_{32}$ Regs[f0]$_{0..31}$
`fsd f0,40(x3)`	Store FP double	Mem[40+Regs[x3]] $\leftarrow_{64}$ Regs[f0]
`sh x3,502(x2)`	Store half	Mem[502+Regs[x2]] $\leftarrow_{16}$ Regs[x3]$_{48..63}$
`sb x2,41(x3)`	Store byte	Mem[41+Regs[x3]] $\leftarrow_{8}$ Regs[x2]$_{56..63}$

Pseudo-instructions are not actual RISC-V instructions but they are often more convenient for the programmer. The assembler converts them to real RISC-V instructions (one or more).

One important pseudo-instruction is `li`, which stands for *load immediate*. As we know this operation is not straightforward in RISC-V because immediates are only 12 bits.

- For small immediates (which fit into 12 bits) the conversion is straightforward:

```
li t0, 4

# translates to:

addi t0, zero, 4
```

- For big immediates (which don't into 12 bits) the conversion needs an upper immediate instruction:

```
li t0, 0xFEDC8765

# translates to:

lui t0, 0xFEDC8     # upper 20 bits
addi t0, t0, 0x765 # lower 12 bits
```

ALU operations are performed on operands held in the processor registers (we also called them register-to-register operations).

Instruction types are:

- Immediate and three-operand instructions;
- Shift instructions;
- Multiply and divide instructions if the M extension is available.

Inst	Name	FMT	Opcode	funct3	funct7	Description (C)	Note
add	ADD	R	0110011	0x0	0x00	rd = rs1 + rs2	
sub	SUB	R	0110011	0x0	0x20	rd = rs1 - rs2	
xor	XOR	R	0110011	0x4	0x00	rd = rs1 ^ rs2	
or	OR	R	0110011	0x6	0x00	rd = rs1 | rs2	
and	AND	R	0110011	0x7	0x00	rd = rs1 & rs2	
sll	Shift Left Logical	R	0110011	0x1	0x00	rd = rs1 << rs2	
srl	Shift Right Logical	R	0110011	0x5	0x00	rd = rs1 >> rs2	
sra	Shift Right Arith*	R	0110011	0x5	0x20	rd = rs1 >> rs2	msb-extends
slt	Set Less Than	R	0110011	0x2	0x00	rd = (rs1 < rs2)?1:0	
sltu	Set Less Than (U)	R	0110011	0x3	0x00	rd = (rs1 < rs2)?1:0	zero-extends

Some interesting pseudo-instructions we note are `seqz` and `snez`:

- `seqz` sets a destination register if the source register is equal to 0:

```
seqz x1,x2

# translates to:

sltiu x1, x2, 1 # if (x2 == 0) x1 <- 1
                # else x1 <- 0
```

- `snez` sets a destination register if the source register is not equal to 0:

```
seqz x1,x2

# translates to:

sltu x1, x0, x2 # if (x2 != 0) x1 <- 1
                # else x1 <- 0
```

Jumps: these comprise control transfer instructions (which are mainly unconditional), including:

- PC-relative jumps;
- Jump to register;
- Conditional branches.

jal	Jump And Link	J	1101111			rd = PC+4; PC += imm	
jalr	Jump And Link Reg	I	1100111	0x0		rd = PC+4; PC = rs1 + imm	

- `j label`: a PC relative jump having a $\pm$ 1MiB range. It is a pseudo-instruction:

```
j label

# translates to:

jal x0, label
```

- `jal label`: a PC relative jump and link having a $\pm$ 1MiB range. It is a pseudo-instruction which assumes `x1` to be the return address:

```
jal label

# translates to:

jal x1, label
```

- `jal x3, label`: a PC relative jump and link having a $\pm$ 1MiB range;
- `jr x3`: an indirect jump to register with $\pm$ 2KiB plus register range. It is a pseudo-instruction:

```
jr x3

# translates to:

jalr x0, 0(x3)
```

- `jalr x3`: an indirect jump to register with $\pm$ 2KiB plus register range. It is a pseudo-instruction which assumes `x1` to be the return address:

```
jalr x3

# translates to:

jalr x1, 0(x3)
```

- `jalr x3, x2, 0`: an indirect jump and link to register. Saves the return value in the destinatino register.
- `call sub1`: calls a named subroutine. It is a pseudo-instruction which assumes `x1` to be the return address:

```
call sub1

# translates to:

auipc x1, sub1 # top 20 bits of subroutine address
jalr x1, offset (x1)
```

- `ret`: returns from a subroutine. It is a pseudo-instruction which assumes `x1` to be the return address:

```
ret

# translates to:

jalr x0, 0(x1)
```

4 Lesson 29-09-26

Branches: we have to quickly finish up on branching instructions in the RISC-V ISA, which we left behind on the past lesson.

These are a special class of instructions: range of $\pm$ 4 KiB and a 12-bit immediate value.

beq	Branch ==	B	1100011	0x0		if(rs1 == rs2) PC += imm	
bne	Branch !=	B	1100011	0x1		if(rs1 != rs2) PC += imm	
blt	Branch <	B	1100011	0x4		if(rs1 < rs2) PC += imm	
bge	Branch $\geq$	B	1100011	0x5		if(rs1 >= rs2) PC += imm	
bltu	Branch < (U)	B	1100011	0x6		if(rs1 < rs2) PC += imm	zero-extends
bgeu	Branch $\geq$ (U)	B	1100011	0x7		if(rs1 >= rs2) PC += imm	zero-extends

The offset must be a multiple of 2, and it is added to PC on positive comparisons. For this, branch instructions compare two registers Rs1 and Rs2

The types are a bit limited:

- `beq` and `bne` take the branch if `rs1` and `rs2` are equal or unequal;
- `blt` and `bltu` take the branch if `rs1` is less than `rs2`;
- `bge` and `bgeu` take the branch if `rs1` is greater than or equal to `rs2`;
- `bgt`, `bgtu`, `ble`, and `bleu` can be synthesized by reversing the operands to `blt`, `bltu`, `bge`, and `bgeu`. The compiler does this automatically.

Some zero-based pseudo-instructions are availale:

- `beqz rs, label`: branches if `rs = 0`:

```
beqz x3, label

# translates to:

beq x3, x0, label
```

- `bnez rs, label`: branches if `rs != 0`:

```
bnez x3, label

# translates to:

bne x3, x0, label
```

- `bltz rs, label`: branches if `rs < 0`:

```
bltz x3, label

# translates to:

blt x3, x0, label
```

- `bgtz rs, label`: branches if `rs1 >= 0`:

```
bgtz x3, label

# translates to:

bgt x3, x0, label
```

4.1 Pipelining

By studying RISC-V, we figured out that the CPU reads a sequence of fixed size instructions from memory, executing them one by one. We know introduce an implementation technique whereby multiple instructions are overlapped in execution: we call this technique **pipelining**. In a pipeline, different units (called pipe *stages* or *segments*) are (at any given time) completing different parts of different instructions in parallel.

Different stages of the pipeline perform different fixed functions on instructions. For instance, we might have (in an undefined implementation) that the execution of each instruction may be composed of at most five clock cycles:

- Instruction fetch cycle (IF)
- Instruction decode/register fetch cycle (ID)
- Execution/effective address cycle (EX)
- Memory access/branch completion cycle (MEM)
- Write-back cycle (WB).

4.1.1 Pipeline metrics

Let's define some *metrics*.

The **throughput** of a pipelined processor is the number of instructions which exit the pipeline in the time unit.

All the pipeline stages are synchronized (they proceed to executing a new task all together): the time for executing one step is called **machine cycle**, and normally corresponds to one *clock cycle*. The length of the machine cycle is determined by the slowest stage of the pipeline.

Lastly, we define **CPI** as *Clock Cycles per Instruction*.

In an ideal pipeline, all the stages are perfectly balanced (i.e., they require the same time). The throughput of an ideal pipeline (i.e., the number of instructions completed in a given time period) is:

$$\text{throughput}_{\text{pipelined}} = \text{throughput}_{\text{unpipelined}} * n$$

where n is the number of stages.

4.1.2 Pipeline performance

Pipelining increases the processor *throughput* without making single instructions execution faster. In fact, single instruction processing is made slower due to the pipeline control *overhead*. The depth of a pipeline is limited by:

- The need for balanced stages;
- Pipelining overhead (pipeline register delay and clock skew).

4.2 Pipeline hazards

Hazards are situations that prevent an instruction from executing during its designated clock cycle. There are three classes of hazards:

- **Structural hazards**: coming from resource conflict;
- **Data hazards**: an instruction depends on the result of a previous instruction;
- **Control hazards**: depend on pipelining branches and other instructions that change the PC.

Let's see ways to deal with these hazards.

4.2.1 Stalling

One way of dealing with hazards is to force the pipeline to **stall**, i.e., to block instructions for one or more clock cycles.

When an instruction is stalled:

- The instructions following the stalled instruction are also stalled;
- The instructions preceding the stalled instruction continue.

This can be easily achieved by modifying the buffer registers, making them able to *conserve* the previous value (this will be the sstage which caused the hazard), and introducing an empty instruction to propagate onwards.

A stall causes the introduction of a *"bubble"* in the pipeline, which has to propagate through the whole pipeline.

This is the simplest way to solve pipeline hazards, as waiting it out (eventually) solves all problems.

4.2.2 Structural hazards

Structural hazards may happen when some pipeline unit is not able to execute all the operations scheduled for a given cycle. This is usually tied to limited resources in the processor.

Some common examples are:

- A given unit is not able to complete its task in one clock cycle;
- The pipeline owns only one register-file write port, but there are cycles in which two register writes are required;
- The pipeline refers to a single-port memory, and there are cycles in which different instructions would like to access to the memory together.

Solution

Removing structural hazards requires adding new hardware or improving the existing one. The designer has to trade-off between performance (add more resources or keep stalling) and cost (add more resources or keep the ones you have), basing on the frequency of occurrence of structural hazards.

This calls for optimization. Let's consider *load structural hazards*: these happen when two instructions contemporarily try to make a memory access to a single-port memory. The clear solution is to split instruction and data memory.

Assume that:

- 40% of instructions access the memory;
- The machine with the structural hazard has a clock 1.05 times faster than the one without.

We want to measure the performance gain.

- For the machine without structural hazard:

$$\text{Average Instruction Time}' = CPI \times T_C$$

  where T_C is the clock period.
- For the machine with structural hazard:

$$\text{Average Instruction Time}'' = CPI_{\text{ideal}} \times T_c'' + \text{Inst. freq} \times \text{Stall penalty}$$

  computed using $T_C'' = T_C \times 1.05$.

4.2.3 Data hazards

Overlapping the execution of instructions, as it is done by pipelining, changes the order of read/write accesses to operands: This can result in:

- Wrong results;
- Undeterministic behavior.

We call these **data hazards**.
Let's consider the code fragment:

```
add r1,  r2, r3  # r1 write

sub r4,  r1, r5  # r1 read from here onwards !
and r6,  r1, r7
or  r8,  r1, r9
xor r10, r1, r11
```

The problem is that the ADD instruction writes the result in R1 in the WB stage.

- The SUB instruction reads R1 in its ID stage, two clock cycles before the one in which R1 is made available (one to feed R1 to WB and one to let WB finish): this produces a wrong result;
- The AND instruction reads R1 in its ID stage, one clock cycle before the one in which R1 is made available: this produces a wrong result;
- The OR instruction reads R1 in its ID stage, in the same clock cycle in which R1 is made available: this produces a correct result, provided that the write accesses to the register file are made before the read accesses. From now ownrads, until we write R1 again,the results are correct (XOR is correct).

Another scenario in which we encounter data hazards is because of *interrupt effects*. If an interrupt occurs during the execution of a critical piece of code (from the point of view of data hazards), correctness may actually be restored (as instructions leave the pipeline as the interrupt is handled). This may cause an undeterministic behavior.

Solution

The typical solution to data hazards is that of **forwarding**. A special hardware in the datapath detects when a previous ALU operation should write the register corresponding to the source of the current ALU operation. In this case, the hardware selects the ALU result as the ALU input rather than the value from the register file.

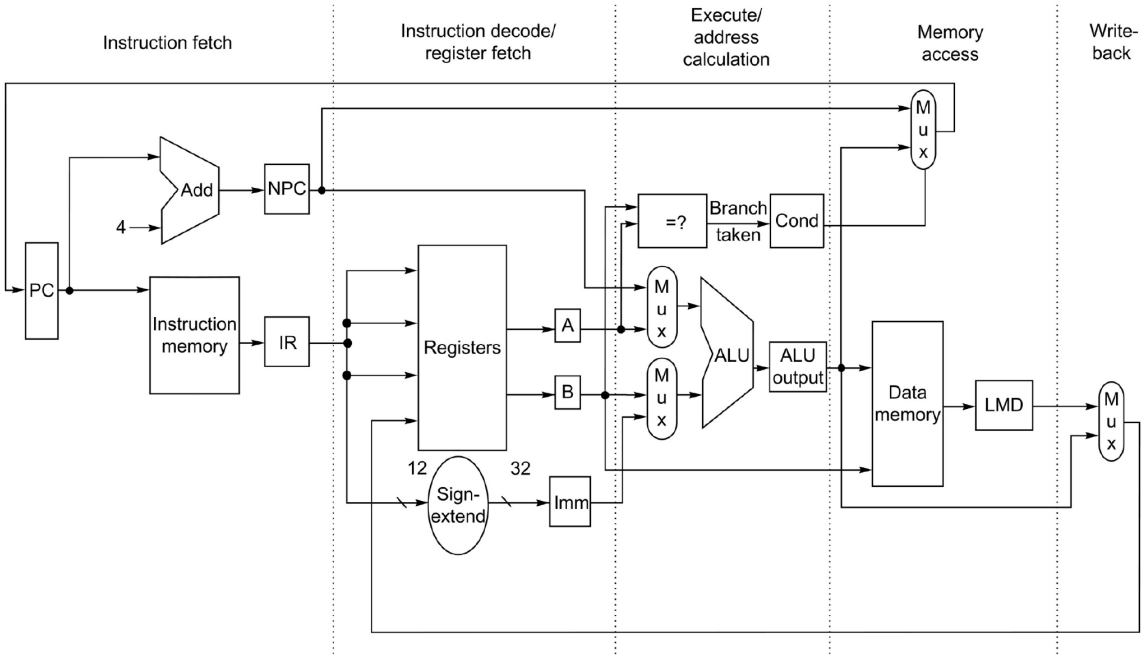
For this to be implemented, the hardware must be able:

- To forward a data from any of the previously started instructions (provided that they didn't already write the data in its final location);
- Not to forward anything, if the following instruction is stalled, or an interrupt occurred.

Forwarding can also be **generalized**: in order to always avoid stalling, it should be made possible between any pipeline register to any input of any functional unit.

4.3 Datapath model

Let's look at a simplified **datapath** schematic to explain pipelines with more detail.



4.3.1 Instruction fetch

The **instruction fetch** stage has the job of fetching the next instruction from program memory and feeding it to the next stage.

As we can see, we use the **PC** (*Program Counter*) to index instruction memory and put the instruction data into the **IR** (*Instruction Register*). Then we increment the program counter by 4 bytes (the size of a RISC-V instruction) and put that into an **NPC** (*New Program Counter*) register.

4.3.2 Instruction decode

We then have to **decode** the instruction.

A number of fields are extracted from the instruction encoding (we saw that RISC-V is highly regular). This allows for *fixed-field decoding*: decoding can be performed while registers are read.

- Parts of the instruction that index *registers* index the **register file**. In this example we index two registers, A and B;
- The immediate operand is also extracted from the instruction encoding, sign extended and sent to the next stage.

We can say that this stage performs both **decoding** and **register fetching**.

4.3.3 Instruction execution

At this point we can **execute** the instructions.

This is performed through the **ALU** (*Arithmetic Logic Unit*), which is basically a big multiplexer of different computation circuits.

We also have to distinguish between input registers and immediates (through multiplexers). Let's look at a couple of instruction types as an example:

- *Memory reference*: ALU output $\leftarrow A + Imm$. Note that we store the target *address* of the memory access! For loads we will use this directly, for stores we will use B and write the ALU output;
- *Register-Register ALU instruction*: ALU output $\leftarrow A$ funct B
- *Register-Register ALU instruction*: ALU output $\leftarrow A$ funct Immediate
- *Branch*: ALU output $\leftarrow NPC +$ Immediate $<< 2$, $Cond \leftarrow (A == B)$. Note that these are 2 operations!

Another thing to note is if the instruction is a branch one: if it is, we have to evaluate a condition and use it to multiplex between NPC and a calculated value (here the value of register A).

We can say that this stage performs both **execution** and **address calculation**.

4.3.4 Memory access

An extra stage is given by the **memory access** stage.

- If a (**load**) instruction requires it, we use the ALU output to address memory and load the needed contents in the **LMD** (*Loaded Memory Data*) register;
- If a (**store**) instruction requires it, we use the B register to address memory, and write the ALU output to it.

This is also the stage where we perform the *branch completion cycle*, that is decide if we should update the NPC or keep it at PC $+ 4$.

More clearly:

- *Memory reference*: $LMD \leftarrow$ Mem[ALUOutput] (loads) or Mem[ALUOutput] $\leftarrow B$ (stores);
- *Branch*:

$$cond\ ?\ PC \leftarrow \text{ALU output},\ \text{otherwise}\ PC \leftarrow NPC$$

4.3.5 Write-back cycle

The last step to perform is the **write-back**.

Based on the instruction type, we update the register file with the obtained result:

- *Register-to-register* or *register-to-memory* instructions feed the ALU output to the register file;
- *Load* instructions feed the LMD to the register file.

More clearly:

- *Register-Register ALU instruction*: Regs$[IR11.7] \leftarrow$ ALU output;
- *Register-Immediate ALU instruction*: Regs$[IR11.7] \leftarrow$ ALU output;
- *Load instruction*: Regs$[IR11.7] \leftarrow LMD$;

In this discussion we skipped a lot of details about the implementation, for instance how the multiplexers are driven, etc... We assume these details to be (more or less) easy to synthesize as combinatorial or sequential networks once an actual architecture is defined.

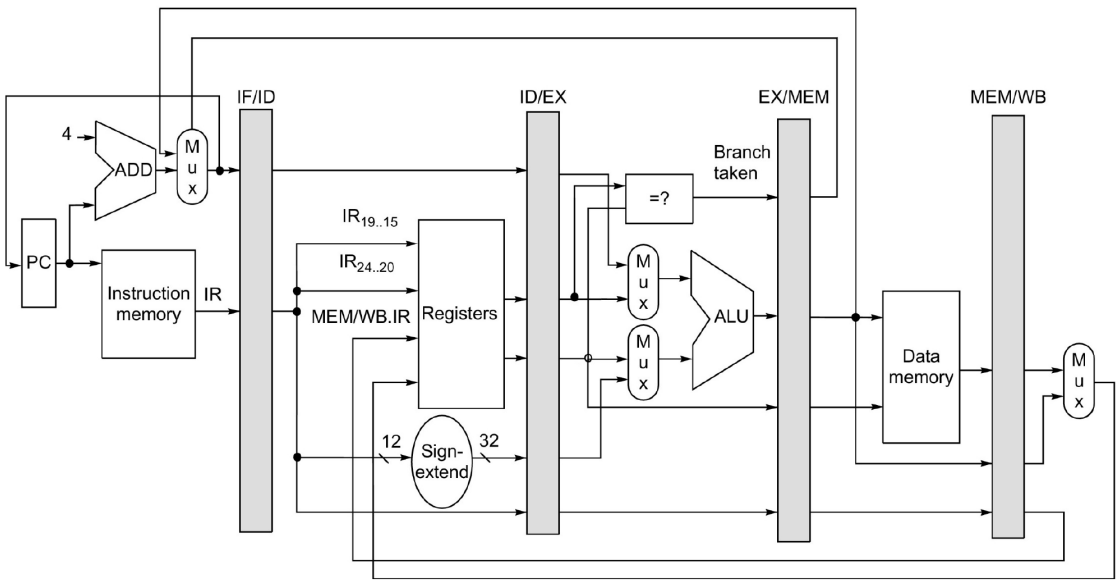
In the described architecture, all instructions require 5 clock cycles, except branch instructions, which require 4 clock cycles. Optimizations could be done for reducing the average CPI: as an example, the ALU instructions could be completed during the memory access cycle. Also, hardware resources could be optimized by avoiding duplications

(e.g., for ALUs, and memory). An alternative single-clock architecture (i.e., executing an instruction per clock cycle) can also be considered, but in that case a simple control unit is required to produce the signals required by the datapath.

4.3.6 Pipelined version

A different solution can be using a **pipelined** version, where a new instruction is started at each clock cycle. Different resources work on different instructions at the same time, and at every clock cycle, each resource can be used for one purpose, only. This means that:

- Separate instruction and data memories (i.e., caches) must be used;
- The register file is used in two stages: for reading (second half of the cycle) in ID and for writing (first half of the cycle) in WB. It must be designed to satisfy these requests during the same clock cycle.
- The PC must be changed in the IF stage, and branch handling must be considered;
- *Pipeline registers* must be added between stages to act as buffers.



4.3.7 Structural hazard control

We don't go in depth on **structural hazard** control as we saw that it depends on the resources available to the processor. The solution remains *stalling*.

4.3.8 Data hazard load control

Let's first go over the **causes** of data hazards.

Causes

A hazard is created whenever there is dependence between instructions, and they are close enough that the overlap caused by pipelining would change the order of access to an operand. In general, this can happen for:

- **Register operands**;

- **Memory operands**: this is possible if:
  - Accesses to memory by load and store are not made in the same stage;
  - Execution can proceed while an instruction waits for a cache miss to be solved.

A problem is that not all data hazards can be solved through data forwarding. Focusing on **loads**, let's consider the code fragment:

```
ld  r1, 0(r2)  # r1 loaded

sub r4, r1, r5 # r1 used
and r6, r1, r7
or  r8, r1, r9
```

Here, when `sub` enters the execution stage, the data memory access is just being initiated on instruction `ld`. The forwarding path here goes back in time, therefore it is impossible to implement. This means it's impossible to forward: our only solution is a stall.

Load control

Implementing the **control** for these bubbles requires that at each clock cycle:

- All tests for detecting a possible data hazard concerning an instruction are performed when this is in the ID stage
- If a data hazard is detected, two actions can be alternatively taken:
  - The appropriate forwarding is activated;
  - The instruction is stalled before entering the stage where operands are not available (i.e., before being issued).

The control circuitry used to handle loads is called **LID** (*Load Interlock Detection*).
Let's look at a table of possible hazard conditions:

Situation	Example fragment	Action

No dependence	`LD R1,45(R2)`
	`DADD R5,R6,R7`
	`DSUB R8,R6,R7`
	`OR R9,R6,R7`
		No hazard possible because no dependence exists on R1 in the immediately following three instructions.

Dependence requiring stall	`LD R1,45(R2)`
	`DADD R5,R1,R7`
	`DSUB R8,R6,R7`
	`OR R9,R6,R7`
		Comparators detect the use of R1 in the DADD and stall the DADD (and DSUB and OR) before the DADD begins EX.

Dependence overcome by forwarding	`LD R1,45(R2)`
	`DADD R5,R6,R7`
	`DSUB R8,R1,R7`
	`OR R9,R6,R7`
		Comparators detect use of R1 in DSUB and forward result of load to ALU in time for DSUB to begin EX.

Dependence with accesses in order	`LD R1,45(R2)`
	`DADD R5,R6,R7`
	`DSUB R8,R6,R7`
	`OR R9,R1,R7`
		No action required because the read of R1 by OR occurs in the second half of the ID phase, while the write of the loaded data occurred in the first half.

We deduce that the following checks have to be performed when a load instruction is in the EX stage and another instruction exploiting the loaded value is in the ID stage.

If operands match, a data hazard is detected and as a result, the control unit must insert a pipeline stall and prevent the instructions in IF and ID stages from advancing.

ID Opcode	IF Opcode	Match
Load	Register-Register ALU	ID/EX.IR[rd] == IF/ID.IR[rs1]
Load	Register-Register ALU	ID/ EX.IR[rd] == IF/ID.IR[rs2]
Load	Load, store, ALU immediate, branch	ID/EX.IR[rd] == IF/ID.IR[rs1]

At this point, given an instruction currently in the ID stage, introducing a stall in the EX stage can be done by:

- Forcing a nop instruction in the ID/EX pipeline register (in RISC-V the nop corresponds to an `addi x0, x0, 0`);
- Forcing the IF/ID pipeline register to maintain the current value (*conserve*);
- Freezing the PC (maintain unaltered IF status).

4.3.9 Data hazard forwarding control

Note that the above table only pertains to *load* operations, that is situations when we are *loading* some data from memory and an instruction wants to access that data while the load is still happening in the pipeline.

We said that there can also be *register* hazards, so now let's shift our attention to a method of solving them, that is **forwarding**. This being said, in a special case (row 4 of the table at 4.3.8) load instructions can also benefit from forwarding from MEM to EX.

SO to summarize, forwarding can be implemented:

- From the ALU or data memory output;
- To ALU inputs, data memory inputs, or the zero detection unit (branch instructions).

The forwarding logic must compare:

- The destination fields of the IR contained in the EX/MEM and MEM/WB registers;
- The source fields of the IR contained in the IF/ID, ID/EX and EX/MEM registers.

We see one final table regarding the forwarding conditions and the *comparisons* (checks):

Source	Opcode	Dest.	Opcode	Forward	Comparison
EX/MEM	Reg–reg ALU, ALU imm.	ID/EX	Reg–reg ALU, ALU imm., load, store, branch	Top ALU	`EX/MEM.rd == ID/EX.rs1`
EX/MEM	Reg–reg ALU, ALU imm.	ID/EX	Reg–reg ALU	Bottom ALU	`EX/MEM.rd == ID/EX.rs2`
MEM/WB	Reg–reg ALU, ALU imm., load	ID/EX	Reg–reg ALU, ALU imm., load, store, branch	Top ALU	`MEM/WB.rd == ID/EX.rs1`
MEM/WB	Reg–reg ALU, ALU imm., load	ID/EX	Reg–reg ALU	Bottom ALU	`MEM/WB.rd == ID/EX.rs2`

4.3.10 Summary of RISC-V dependency handling

To summarize, we note that the following situations might happen in the pipeline, and the tools we have available to solve them:

Situation	Solution
No dependence	Do nothing
Structural hazard	Stall
Any source IF/ID register in load target ID/EX stage	Stall (table 1)
Any reg-reg ID/EX register in any target EX/MEM stage	Forward (table 2)
Any reg-reg or load ID/EX register in any target MEM/WB stage	Forward (table 2)

Exact register comparisons are found in the tables above. Table 1 refers to the *load control* table, wherease table 2 refers to the *forwarding control* table. It's clear that we are forced to stall only on structural and load dependencies (so when the data physically isnt in the CPU at the needed time).

For clarity (and also for fun) we test these out on a microarchitectural simulator. The simulator chosen is **Gem5** (`https://www.gem5.org/`) as it's used by the course.

A code example is provided, and underneath the disassembly without pseudo-instructions and pipeline state. It should be clear that the hazards in table 1 (section 4.3.8) are solved via stalling, whereas the hazards in table 2 (section 4.3.9) are solved, if the processor supports it, via forwarding.

1. **Table 1** (4.3.8), **type 1**: ALU source register 1 target of load.

```
.section .data
    val: .word 42

# The text section contains the instructions that the CPU runs.
.section .text
# Make _start visible as the point where the program begins.
.globl _start
_start:

# setup load
    la t0, val

# perform test a
    lw t1, 0(t0)
    add t5, t1, t2

# The End block stops the program and returns control to the simulator.
End:
    li a0, 0
    li a7, 93
    ecall
```

PC Address	Instruction	Control / cache	1	2	3	4	5	6	7	8	9
0x10094	auipc x5, 1		F	D	E	M	W				
0x10098	addi x5, x5, 28			F	D	E	M	W			
0x1009c	lw x6, 0(x5)				F	D	E	M	W		
0x100a0	add x30, x6, x7					F	D	S	E	M	W

2. **Table 1** (4.3.8), **type 2**: ALU source register 2 target of load.

```
.section .data
    val: .word 42
```

```
3
4  # The text section contains the instructions that the CPU runs.
5  .section .text
6  # Make _start visible as the point where the program begins.
7  .globl _start
8  _start:
9
10 # setup load
11     la t0, val
12
13 # perform test a
14     lw t1, 0(t0)
15     add t5, t2, t1
16
17 # The End block stops the program and returns control to the simulator.
18 End:
19     li a0, 0
20     li a7, 93
21     ecall
```

PC Address	Instruction	Control / cache	1	2	3	4	5	6	7	8	9
0x10094	auipc x5, 1		F	D	E	M	W				
0x10098	addi x5, x5, 28			F	D	E	M	W			
0x1009c	lw x6, 0(x5)				F	D	E	M	W		
0x100a0	add x30, x7, x6					F	D	S	E	M	W

3. **Table 1** (4.3.8), **type 3**: load source target of (other) load.

```
1  .section .data
2  val:  .byte 42
3  addr: .word val
4
5  # The text section contains the instructions that the CPU runs.
6  .section .text
7  # Make _start visible as the point where the program begins.
8  .globl _start
9  _start:
10
11 # setup load
12     la t1, addr
13
14 # perform test c
15     lw t2, 0(t1)
16     lb t3, 0(t2)
17
18 # The End block stops the program and returns control to the simulator.
19 End:
20     li a0, 0
21     li a7, 93
22     ecall
```

PC Address	Instruction	Control / cache	1	2	3	4	5	6	7	8	9
0x10094	auipc x6, 1		F	D	E	M	W				
0x10098	addi x6, x6, 29			F	D	E	M	W			
0x1009c	lw x7, 0(x6)				F	D	E	M	W		
0x100a0	lb x28, 0(x7)					F	D	S	E	M	W

4. **Table 2** (4.3.9), **type 1**: ALU source register 1 target of ALU.

```
# The text section contains the instructions that the CPU runs.
.section .text
# Make _start visible as the point where the program begins.
.globl _start
_start:

# setup registers
    li t2, 2
    li t3, 2
    li t5, 2
    nop

# perform test a
    add t1, t2, t3
    sub t4, t1, t5

# The End block stops the program and returns control to the simulator.
End:
    li a0, 0
    li a7, 93
    ecall
```

PC Address	Instruction	Control / cache	1	2	3	4	5	6	7	8	9	10
0x10074	addi x7, x0, 2		F	D	E	M	W					
0x10078	addi x28, x0, 2			F	D	E	M	W				
0x1007c	addi x30, x0, 2				F	D	E	M	W			
0x10080	addi x0, x0, 0					F	D	E	M	W		
0x10084	add x6, x7, x28						F	D	E	M	W	
0x10088	sub x29, x6, x30							F	D	E	M	W

5. **Table 2** (4.3.9), **type 1**: ALU source register 2 target of ALU.

```
# The text section contains the instructions that the CPU runs.
.section .text
# Make _start visible as the point where the program begins.
.globl _start
_start:

# setup registers
    li t2, 2
    li t3, 2
    li t5, 2
    nop

# perform test a
    add t1, t2, t3
    sub t4, t5, t1

# The End block stops the program and returns control to the simulator.
End:
    li a0, 0
    li a7, 93
    ecall
```

PC Address	Instruction	Control / cache	1	2	3	4	5	6	7	8	9	10
0x10074	addi x7, x0, 2		F	D	E	M	W					
0x10078	addi x28, x0, 2			F	D	E	M	W				
0x1007c	addi x30, x0, 2				F	D	E	M	W			
0x10080	addi x0, x0, 0					F	D	E	M	W		
0x10084	add x6, x7, x28						F	D	E	M	W	
0x10088	sub x29, x30, x6							F	D	E	M	W

6. **Table 2** (4.3.9), **type 1**: ALU source register 1 target of load (delayed).

```
.section .data
val: .byte 3

# The text section contains the instructions that the CPU runs.
.section .text
# Make _start visible as the point where the program begins.
.globl _start
_start:

# setup load
    la t0, val
    li t6, 2
    nop

# perform test c
    lb t1, 0(t0)
    nop # or anything else that doesn't touch t1
    add t5, t1, t6

# The End block stops the program and returns control to the simulator.
End:
    li a0, 0
    li a7, 93
    ecall
```

PC Address	Instruction	Control / cache	1	2	3	4	5	6	7	8	9	10	11
0x10094	auipc x5, 1		F	D	E	M	W						
0x10098	addi x5, x5, 40			F	D	E	M	W					
0x1009c	addi x31, x0, 2				F	D	E	M	W				
0x100a0	addi x0, x0, 0					F	D	E	M	W			
0x100a4	lb x6, 0(x5)						F	D	E	M	W		
0x100a8	addi x0, x0, 0							F	D	E	M	W	
0x100ac	add x30, x6, x31								F	D	E	M	W

7. **Table 2** (4.3.9), **type 1**: ALU source register 2 target of load (delayed).

```
.section .data
val: .byte 3

# The text section contains the instructions that the CPU runs.
.section .text
# Make _start visible as the point where the program begins.
.globl _start
_start:

# setup load
```

```
    la t0, val
    li t6, 2
    nop

# perform test c
    lb t1, 0(t0)
    nop # or anything else that doesn't touch t1
    add t5, t6, t1

# The End block stops the program and returns control to the simulator.
End:
    li a0, 0
    li a7, 93
    ecall
```

PC Address	Instruction	Control / cache	1	2	3	4	5	6	7	8	9	10	11
0x10094	auipc x5, 1		F	D	E	M	W						
0x10098	addi x5, x5, 40			F	D	E	M	W					
0x1009c	addi x31, x0, 2				F	D	E	M	W				
0x100a0	addi x0, x0, 0					F	D	E	M	W			
0x100a4	lb x6, 0(x5)						F	D	E	M	W		
0x100a8	addi x0, x0, 0							F	D	E	M	W	
0x100ac	add x30, x31, x6								F	D	E	M	W

5 Lesson 02-10-26

5.1 Control hazards

We know move on to discussing **control hazards**. We explicitly ignored them until now as they are:

- Slightly more complicated;
- Open up a world of possibilities for CPU optimizations.

They are due to branches (conditional and unconditional), which may change the PC after the following instruction has been fetched already. In the case of conditional branches, the decision on whether the PC should be modified (branch taken) or not (branch untaken) can be taken even later. In the basic RISC-V implementation, the PC is written with the target address (if the jump is taken) at the end of the EXE stage, i.e., 2 clock cycles after the IF of the branch. This behavior delays the target instruction by two clock cycles.

5.1.1 Stalling

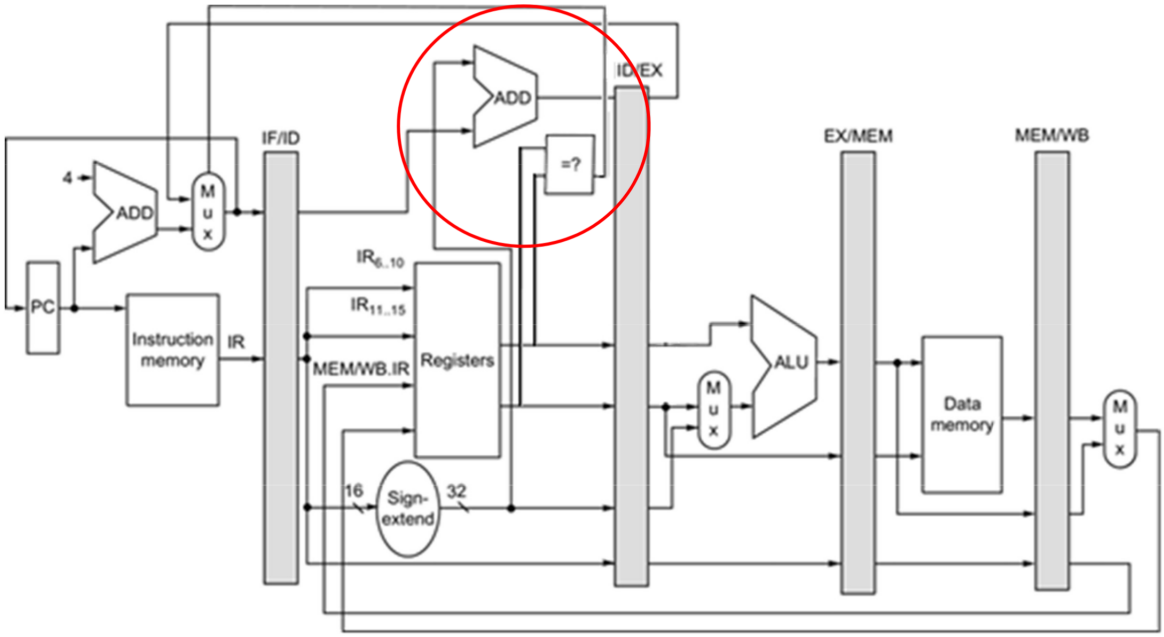
A possible solution is based on stalling the pipeline as soon as a branch instruction is detected (ID stage) by:

- Deciding earlier whether the branch has to be taken or not;
- Computing earlier the new PC value.

This means that we make a temporary decision and continue loading the pipeline as if the branch wasn't there (or as if it was always taken, depending on the prediction). If the branch does, in fact, get taken (or not taken) then we wasted clock cycles and have to flush the pipeline. Otherwise, we win.

5.1.2 Stalling with modified pipeline

This clearly has an advantage in modifying the pipeline for early branch address computation (in the ID stage instead of the EX stage).



Note that this approach has an issue: at most we can put the branch prediction in the ID (decode) stage, that is the second stage of the branch instruction processing. This means that the instruction loaded in the pipeline right after the branch one is always wasted (never depends on the branch prediction).

We show a figure that details this situation.

Branch instruction	**IF**	**ID**	**EX**	**MEM**	**WB**		
Branch +1		**IF**	**--**	**--**	**--**	**--**	
Branch Target			**IF**	**ID**	**EX**	**MEM**	**WB**
Branch Target + 1				**IF**	**ID**	**EX**	**MEM**

From this, we introduce the fact that several techniques can be used to manage branches:

- Freezing the pipeline;
- Predict untaken;
- Predict taken;
- Delayed branch.

5.1.3 Freezing the pipeline

This is the previously proposed solution: the pipeline is stalled (or flushed) as soon as a branch instruction is detected, and until the decision about the branch is known. It is the simplest solution to implement.

It wastes 2 clock cyceles (without the modified pipeline), or at least 1 (with the modified pipeline, decode is second stage).

5.1.4 Predict untaken

This technique introduces a *guess*. The *predictions* we talked about so far weren't predictions in the strict sense: at most we were making the actual condition calculation earlier (in the ID stage), but we still were sure about actual branches taken. Now we assume an arbitrary value of the condition before evaluating it:

- We assume the branch is not taken;
- We avoid any change in the pipeline status until the branch decision has been taken;
- We undo all the performed operations if the branch turns out to be taken.

Considering our last solution, this makes the most sense as we loaded the instruction after the branch anyways: if the branch isn't taken, there's no point in flushing.

Untaken branch instruction	IF	ID	EX	MEM	WB				
Instruction $i + 1$		IF	ID	EX	MEM	WB			
Instruction $i + 2$			IF	ID	EX	MEM	WB		
Instruction $i + 3$				IF	ID	EX	MEM	WB	
Instruction $i + 4$					IF	ID	EX	MEM	WB

Taken branch instruction	IF	ID	EX	MEM	WB				
Instruction $i + 1$		IF	**idle**	**idle**	**idle**	**idle**			
Branch target			IF	ID	EX	MEM	WB		
Branch target + 1				IF	ID	EX	MEM	WB	
Branch target + 2					IF	ID	EX	MEM	WB

5.1.5 Predict taken

The same technique can be done by *guessing* the branch is taken. The results are the same, just that we load the instruction after the target (instead of the one after the branch). Performance gains are similar assuming branches get taken or not taken with equal probability.

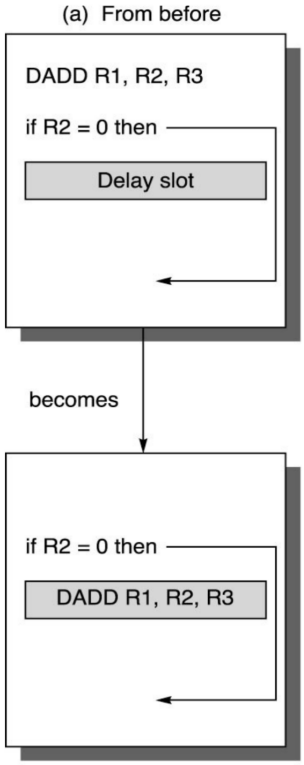
Note the role of the *compiler*: If the hardware supports the predict taken or predict untaken scheme, the compiler can improve performance by generating code which maximizes the chance for the processor to make the right prediction.

For instance: considering the loop implementation, the for scheme is suitable for the predict untaken scheme, the do while for the predict taken. In fact, the for scheme involves executing a conditional branch instruction before the loop body, which results in a not taken branch as many time as the number of iterations. the do while does the opposite.

5.1.6 Delayed branch

This technique is based on filling the slot after the branch instruction (named branch-delay slot) with instructions which have to be executed no matter the branch outcome. It is an actual *architectural* change on the ISA: branches actually branch only after one instruction, and the slot which was wasted before now has a fixed purpose.

It is up to the compiler (or the seasoned programmer) to fill each branch-delay slot with the right instructions. The processor does nothing special when a branch instruction is decoded.



MIPS famously used one branch delay slot. ARM also historically had behavior resembling this in older ARM architectures, where instructions following a branch could already have entered the pipeline, but ARM's architectural rules are different from the classic MIPS "one guaranteed delay-slot instruction" model. Today, other techniques are preferred.

The performance depends on the compiler ability in finding the right instructions to put in the delay slots. We find that using this technique, only about 30% of branches do produce a penalty.

When deeply pipelined processors are used, the delay slots is longer, and the advantages of delayed-branches smaller. Therefore, such RISC architectures do not support delayed branches.

5.2 Multicycle operations

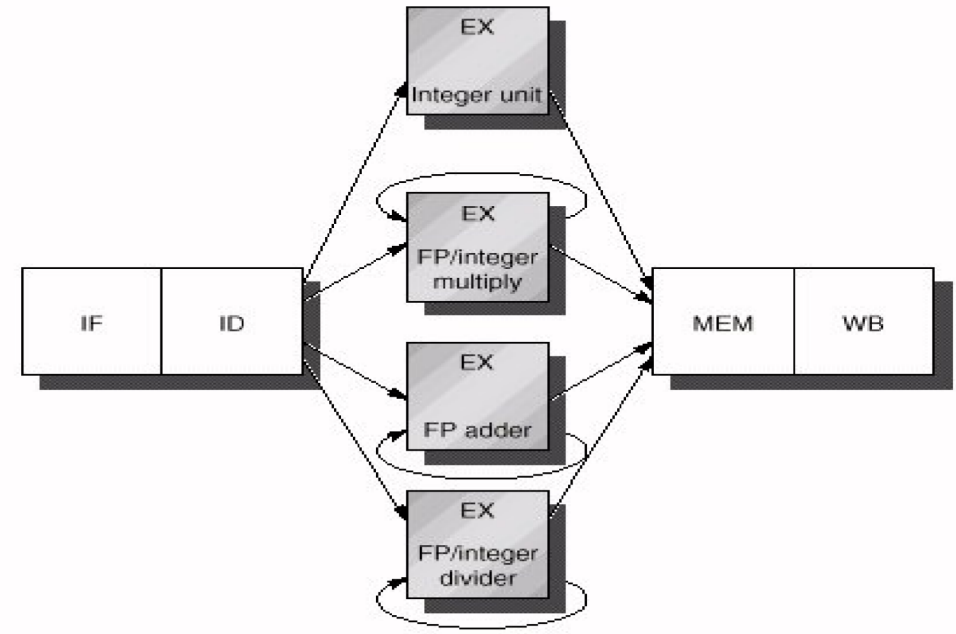
Let's talk *complex operations*, such as **floating point**. Floating point units perform more complex operations than integer ones. Therefore, in order to force them to perform their job in a single clock cycle, the designer should:

- Either use a very slow clock;
- Or make these units very complex.

As a popular alternative, floating point units generally require more than one clock cycle to complete. The EX stage is composed of different functional units, and takes as many clock cycles, as the instruction requires.

5.2.1 Multicycle execution

The resulting pipeline is as follows, with the EX stage divided in multiple units with feedback and (possibly) multicycle instruction permanence.



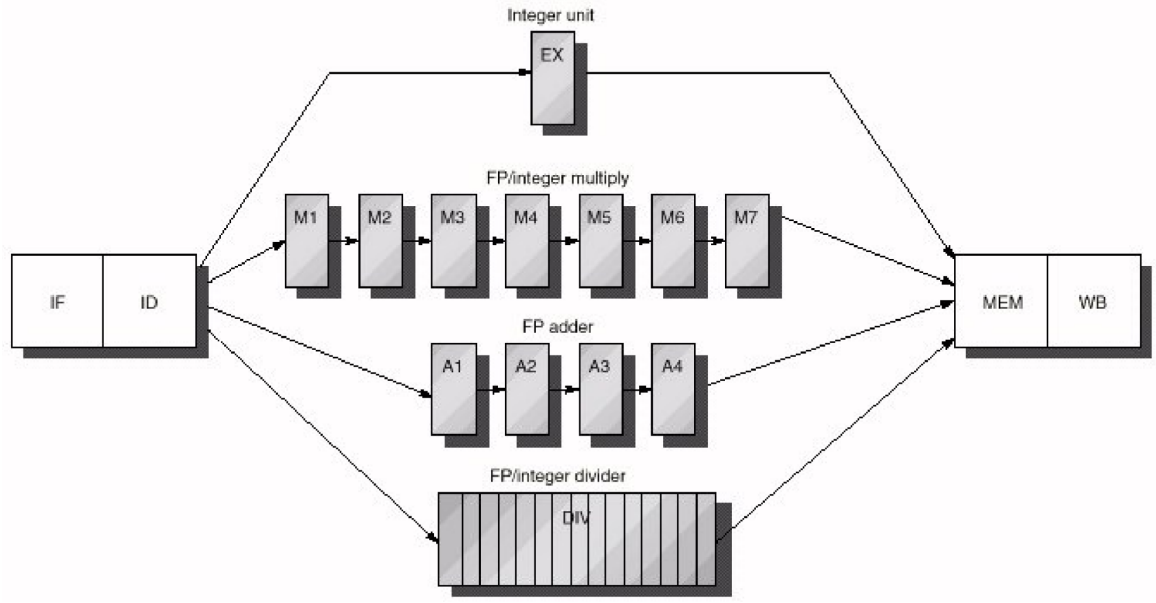
We see the problems of this approach by considering certain *metrics* of the execution phase:

- **Latency**: it is the number of cycles that should last between an instruction that produces a result and an instruction that uses the same result;
- **Initiation interval**: it is the number of cycles that must elapse between issuing two operations of the same type to the same unit.

This approach increases latency as some instructions stay in the execution stage longer.

5.2.2 Multicycle pipelines

An improvement is to *pipeline* our expanded execution stage:



Each instruction, as with all pipelines, stays in the processor the same amount of time (if not longer because of pipelining overhead), but at least we have the increased throughput from more instruction coexisting in the pipeline at the same time.

One thing to note, though, is that due to the different structure of the EX stage, *hazards* may become more frequent.

5.2.3 Multicycle hazards

Let's see the **hazards** we can encounter with this architecture.

- **Structural hazards** can occur in ways we didn't encounter before:
  - Because of the unpipelined divide unit (at least in our example above), several instructions could need it at the same time;
  - Because the instructions have varying running times, the number of register writes required in a cycle can be larger than 1 (and this applies generally).

	Clock cycle number										
Instruction	1	2	3	4	5	6	7	8	9	10	11
`fmul.d f0,f4,f6`	IF	ID	M1	M2	M3	M4	M5	M6	M7	MEM	WB
...		IF	ID	EX	MEM	WB					
...			IF	ID	EX	MEM	WB				
`fadd.d f2,f4,f6`				IF	ID	A1	A2	A3	A4	MEM	WB
...					IF	ID	EX	MEM	WB		
...						IF	ID	EX	MEM	WB	
`fld f2,0(x2)`							IF	ID	EX	MEM	WB

This is particularly bad as in our integer pipeline there was no way to have WB stages overlap.

The solutions, as always (we saw structural hazards on the integer pipeline), are:

- Adding other write ports (normally too expensive)
- Forcing a structural hazard via *stalling*:
  * Instructions are stalled in the ID stage;
  * Or instructions are stalled before entering the MEM or WB stage.

- **Data hazards** can occur, are more frequent and have an higher performance impact.
  - Because of longer latency of operations, stalls for data hazards may stall the pipeline for longer periods.

	Clock cycle number																
Instruction	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
`fld f4,0(x2)`	IF	ID	EX	MEM	WB												
`fmul.d f0,f4,f6`		IF	ID	Stall	M1	M2	M3	M4	M5	M6	M7	MEM	WB				
`fadd.d f2,f0,f8`			IF	Stall	ID	Stall	Stall	Stall	Stall	Stall	Stall	A1	A2	A3	A4	MEM	WB
`fsd f2,0(x2)`					IF	Stall	Stall	Stall	Stall	Stall	Stall	ID	EX	Stall	Stall	Stall	MEM

    As with the before example, the long stall is due to the hefty execution size of the floating point instructions, which keeps the target register "locked" for more than our integer pipeline did.

    This is a case of a **RAW** (*Read After Write*) hazard;
  - The concurrent WB retires are also worth noting: new kinds of data hazards might manifest because multiple instructions reach WB at the same time.

    This is a case of a **WAW** (*Write After Write*) hazard.

    The simple solution is: before issuing an instruction to the EXE stage, check whether it is going to write on the same register of an instruction still in the EXE stage. In this case, stall the new instruction.

5.2.4 Summary of hazards

To sumamrize, iif hazard detection is always performed in the ID stage, three checks have to be performed:

- Structural hazards (involving the divide unit and the write port);
- RAW data hazards: check whether some source register is listed among the destination registers of pending instructions, and whether this register will not be available at the right moment
- WAW data hazards: check whether the instruction currently in ID has the same destination register of any instruction in EXE.

6 Lesson 06-10-26

6.1 Exceptions and interrupts

Exceptions are events that modify the normal execution order of the program.

Usually, to refer to events coming from the outside, we use the term **interrupt**. Interrupts come from I/O devices, or reset requests from the device's hardware. Sometimes we see the name *internal interrupt* used for exceptions.

Possible causes of exceptions are:

- I/O device request
- Operating system call by a user program
- Tracing instruction execution
- Breakpoint (programmer-requested interrupt)
- Integer arithmetic overflow or underflow
- FP arithmetic anomaly
- Page fault
- Misaligned memory accesses
- Memory-protection violation
- Undefined instruction
- Hardware malfunction
- Power failure.

Exceptional events (*exceptions*, *interrupts*, or *faults*) are more complex to handle in pipelined architectures because several instructions are being executed at a time.

6.1.1 Exception taxonomy

There are a few orthogonal characteristics we can use to classify exceptions.

- **Synchronous** vs **asynchronous**: an exception is *synchronous* if it can always be triggered in the same position by the code, *asynchronous* if it comes from an external source;
- User **requestable** vs **coerced**: user *requested* exceptions are similar to procedures. *Coerced* exceptions are out of the control of the user program;
- User **maskable** vs **non-maskable**: *maskable* exceptions can be suppressed by the user, whereas *non-maskable* exceptions need to be serviced in any case;
- **Within** vs **between** instructions: exceptions can be raised *within* an istruction (as its executing), or *between* instructions. In this case they are generated by the instruction itself;
- **Resume** vs **terminate**: after raising an exception, a program can either *resume* execution or *terminate* completely.

  So called *restartable machines* are able to handle an exception, save the state, and restart without affecting the execution of the program. Nearly all processors nowadays are restartable machines.

6.1.2 Pipeline

When an exception occurs, the pipeline must execute the following steps:

- Force a trap instruction into the pipeline on the next IF stage;
- until the trap is taken, turn off all writes for the instruction that raised the exception (i.e., the faulty instruction) and for all the following instructions in the pipeline;
- when the exception-handling procedure receives control, it immediately saves the PC of the faulty instruction.

6.1.3 Interrupt protocols

Let's compare *protocols* for handling interrupts on different architectures.

- On **x86**, the general routine is:
  1. An external device sends an interrupt request;
  2. The CPU detects the interrupt;
  3. The CPU reads the Interrupt Type N from the bus;
  4. The CPU saves the value of the processor status word (PSW) and the return address (Code Segment and Instruction Pointer registers) in the stack;
  5. The CPU resets the interrupt flags disabling external interrupts and trap mode;
  6. Using N as an index, the processor reads from the Interrupt Vector Table the address where to jump;
  7. The CPU jumps to the Interrupt Service Routine.
- On **ARM**, the routine changes to:
  1. An external device sends an interrupt request;
  2. The CPU detects the interrupt
  3. The CPU pushes into the current stack the information of the current stack frame composed by 8 registers including the Program Counter and the Processor Status Register;
  4. The CPU updates the processor flags and jumps to the address given by the interrupt type N: on that position a new jump to the actual Interrupt Service Routine may be placed.

For RISC-V, for now let's not focus on interrupt protocol but on possible **causes** of exceptions during each stage of the pipeline.

Stage	Cause
IF	Pagefault on instruction fetch, Misaligned access, Memory-protection violation
ID	Undefined or illegal opcode
EX	Arithmetic exception
MEM	Page fault on data fetch, Misaligned memory access, Memory-protection violation
WB	None.

The fact that all RISC-V instructions write back only in the WB phase (except stores, which write back to memory during the MEM phase) makes pipeline stalling on exceptions easier. As we saw in 1.2.5, ARM addressing allows for *pre-indexing* and *post-indexing*, which access registers earlier than the writeback phase. This clearly makes the situation more complex.

6.1.4 Precise exceptions

A processor is said to have **precise exceptions** if the pipeline can be stopped so that:

- The instructions just before the instruction that triggered the exception are completed;
- The instructions following the instruction that triggered the exception can be restarted from scratch.

Restarting after an exception may be really hard if exceptions are not handled precisely. Precise exception handling is a must for most architectures, at least for integer instructions. Of course, the problem of precise exception is that they are more difficult to implement / slower.

The main cause for imprecise exceptions is having slower units, such as a *Floating-Point* unit. The solutions in this case can be:

- Accepting imprecise exceptions;
- Providing a fast, but imprecise operating mode, and a slow, but precise one (only one FP instruction is executed at a time);
- Forcing the FP units to early determine whether an instruction can cause an exception, and issuing further instructions only when the previous ones are guaranteed not to cause an exception;
- Buffering the results of each instruction until all the previously issued instructions have been completed.

6.2 Instruction-Level Parallelism

Pipelines exploit the **parallelism** existing among **instructions** (*Instruction-Level Parallelism*, or **ILP**), which allows their execution in parallel. The highest the amount of ILP that can be found and exploited, the better the performance of the pipeline.

The are 2 main ways to exploit ILP:

- **Dynamic**, depending on the hardware to locate parallelism. This dominates both the desktop and embedded market;
- **Static**, depending on the compiler to locate parallelism before the program is even loaded into memory. Can be found in the embedded market, for instance the ARM Cortex-M8. Intel briefly experimented with it in the *Itanium* (IA-64) architecture.

  The two approaches can be partly combined.

6.2.1 Basics of ILP

Let's first look at the basics of the *static* kind of ILP.

- The first idea is to scan the code for **basic blocks**. The first kind of ILP is the one among instructions belonging to the same basic block. A basic block is a sequence of instructions with:
  - No branches in, except to the entry
  - No branches out, except at the exit.

  Inside a basic block, *rescheduling* can be performed to optimized instruction parallelism. For typical RISC-V programs, the typical size of a basic block is between 4 and 7 instructions. Since these instructions are likely to be dependent one from the other, the amount of parallelism existing within a basic block is normally rather small.
- To further increase the available parallelism, the parallelism among iterations of a **loop** is considered. *Loop unrolling* is the usual technique to avoid branch-related control hazards. Note that unrolling loops by explicitly replicating the loop body multiple times, widens the loop basic block.

  In this way:
  - The relative overhead due to the control of iteration is reduced;

– The loop body is made wider, thus increasing the chance for the compiler to exploit rescheduling to eliminate stalls.

The disadvantage is that loop unrolling increases the size of the code.

Another possible solution is that of **SIMD** (*Single Instruction, Multiple Data*). More precisely, *Single instruction stream, multiple data streams* may be exploited in:

– **Vector processors**: A vector instruction operates on a set of data, instead of on a scalar data (as a normal instruction);
– *Graphics Processing Units* (**GPU**s): Different functional units perform similar tasks in parallel acting on multiple data.

6.3 Dependencies

If two instructions are not dependent, they can be executed in parallel without any stall. If they are dependent, they have to be executed in order (although partly overlapped). Therefore, exploiting the parallelism among instructions requires identifying dependencies existing among them.

There are three kinds of dependencies:

- **Data dependencies**: an instruction j data depends on instruction i if i produces a result that j uses, or there is some transitive chain of instructions which j data depends on and i is a data dependency on;
- **Name dependencies**:
- **Control dependencies**:

At this point we have to make some clarifications. *Dependencies* are properties of the program:

- They create the possibility for a *hazard*;
- Determine the order in which results must be calculated;
- Set an upper bound on the amount of parallelism that can be exploited.

Hazards are potential problems stemming from dependecies in a specific pipeline organization. Stalls depend on the program and the pipeline: a dependency can cause a hazard or not, and the hazard can cause a stall or not (e.g., forwarding can avoid the stall).

6.3.1 Data dependencies

Detecting dependencies involving registers is easy. Detecting dependencies involving memory cells is much more difficult, because accesses to the same cell can look very different. If static techniques are used, the compiler must adopt a conservative approach, assuming that any load instruction refers to the same cell of a previous store. Dependencies involving memory cells can only be detected at run time, when the addresses are known.

6.3.2 Name dependencies

A name dependency occurs when two instructions refer to the same register or memory location (name) but there is no flow of data associated to the name. There are two kinds of name dependencies between an instruction i and an instruction j that follows:

- **Antidependence**: instruction j writes a register or memory location that instruction i reads, and instruction i is executed first;
- **Output** dependence: both instruction i and instruction j write the same register or memory location.

As we know, these dependencies aren't real dependencies and can be eliminated by:

- Increasing the number of registers in the processor;
- Implementing *register renaming*.

7 Lesson 16-10-26

Let's get back to dependencies, which we first introduced in 6.3.

7.0.1 Data hazards

Each time an operand involved in a dependency is accessed in a different order than the original one, there could be a hazard.

This means that the program output may become wrong.

Data hazards can be classified in three categories, based on the type of two instructions i, j executed in order:

- i writes, j reads: **RAW** (*Read After Write*), which stems from actual data dependencies;
- i writes, j writes **WAW** (*Write After Write*) which stems from an output (name) dependency;
- i reads, j writes **WAR** (*Write After Read*) which stems from an antidependence (name).

We can summerize this with a graph:

	i writes	i reads
j writes	**WAW** (output)	**WAR** (antidependency)
j reads	**RAW** (data)	//

7.1 Control dependencies

A control dependency occurs when an instruction depends on a branch.

- An instruction that is control dependent on a branch cannot be moved before the branch (so that its execution is no more controlled by the branch).
- An instruction that is not control dependent on a branch cannot be moved after the branch (so that its execution become dependent on the branch).

Normally, preserving control dependencies is a sufficient condition for preserving the program correctness. But there are cases in which the reverse is not true.

What actually has to stay fixed is:

- *Exception behavior*;
- *Data flow*;

7.1.1 Exception behavior

Exception behavior conservation ensures that any change in the order of instruction execution must not change how exceptions are raised in the program.

Take for instance:

```
add x2, x3, x4
beqz x2, L1
ld x1, 0(x2)

L1: ...
```

The `ld` instruction can raise an exception, so it cannot be moved before the branch, as an exception caused by the load can then happen no matter whether the branch is taken or not.

7.1.2 Data flow

Data flow is the actual flow of data among instructions that produce results and consume them. Data flow must be preserved.

Take for instance:

```
add x1, x3, x4
beqz x2, L1
sub x1, x5, x6

L1:
  or x7, x1 ,x8
```

In this example, the value of `x1` used by the `or` instruction must be that produced by the `add` if the branch is taken, or that produced by the `sub` if it is not taken.

There are cases in which it is possible to violate the control dependence without affecting the exception behavior or the data flow. These are cases where:

- The moved instruction cannot generate exceptions;
- The program results are not changed anyway.

By doing this, the compiler speculates, i.e., bets on the branch not to be taken.

7.2 Dynamic scheduling

We introduced the concept of *Instruction Level Parallellism* (**ILP**) (6.2), with the associated *dependencies*, and even introduced some *static* techniques (6.2.1). Now we look at the *dynamic* side of things, including *out-of-order* execution.

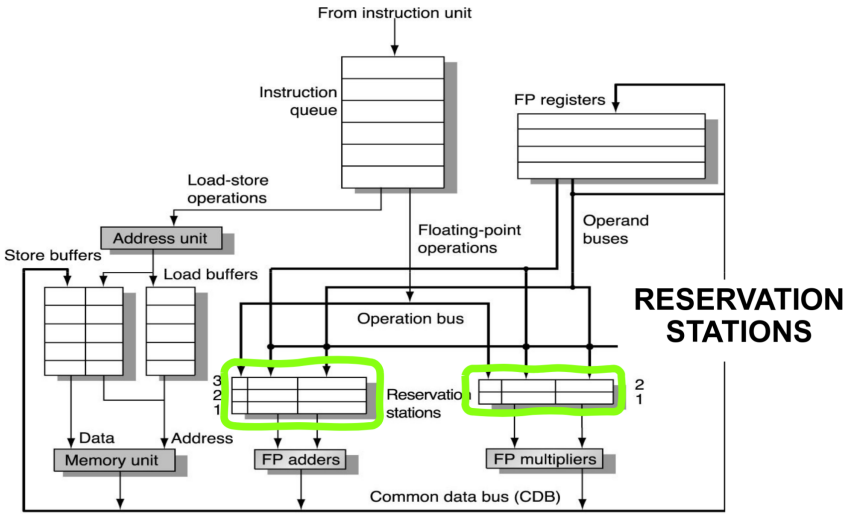
To summarize, dynamic scheduling is a technique aimed at rearranging in hardware the instruction execution order to reduce the pipeline stalls while maintaining data flow and exception behavior.

Assuming the basics of dependencies and out of order execution are covered, we move straight to *Tomasulo*'s algorithm.

7.2.1 Tomasulo's algorithm

Robert Tomasulo was the architect of the IBM 360/91 floating point unit: his ideas were first published in a paper of 1967. The same ideas were then re-used when the first superscalar processors were built. The key ideas in the Tomasulo's algorithm are to:

- Track operand availability;
- Rename registers (which we might have mentioned when talking about name dependencies).



8 Lesson 20-10-26

In the last lesson we saw the structure of the Tomasulo architecture. Now we look into the reservation stations and the register files in more details.

8.0.1 Fields

First off, the reservation station's records are made out of the following *fields*:

Op	V_j	V_k	Q_j	Q_k	A	Busy

These are:

- O_p: represents the operation being performed on operands;
- Q_j, Q_k: the reservation station that will produce the relevant source operand (0 indicates the value is in V_j, V_k);
- V_j, V_k: the value of the source operands;
- A: used to hold the memory address information for a load or store;
- Busy: 1 if occupied, 0 if not occupied.

Information about which register is where is contained within the register file (floating point or integer): we store a Q_i flag, which records the reservation station whose result should be stored in this register (if blank or 0, no values are destined for this register).

8.0.2 Loads and stores

Load and store instructions can be executed in any order, provided that they do not refer to the same address. In this case, by exchanging their order:

- If the load is before the store, a WAR hazard may happen;
- If the store is before the load, a RAW hazard may happen.

To avoid hazards coming from reordering load and store instructions, the processor must:

- Compute memory addresses in order;
- Perform some check for every load or store instruction already in the buffers.

More formally:

- When a **load** is about to be issued, the store buffer is checked for store instructions acting on the same address, and the load is not issued until these previous instructions complete;
- When a **store** is about to be issued, both the load and store buffers are checked for load and store instructions acting on the same address, and the load is not issued until these previous instructions complete;

8.1 Speculation

This section follows pretty naturally from what we've seen so far:

- First off we introduced the bare, unpipelined, processor;
- Then we introduced the *pipeline*, and saw that hazards appear;
- When more than one execution unit was needed, we issued to pipelined stages via an issue unit in the *Tomasulo architecture*. This showed that hazards were really dependencies showing up has hardware constraints.

We know extend the Tomasulo architecture by introducing **speculation**.

So far, when we met a branch instruction in the Tomasulo architecture, we stopped issuing instruction until the branch was taken. We saw that when pipelining some kind of speculation could have been done on the branch outcome. Let's see if we can do the same here.

Hardware-based speculation is a technique for reducing the effects of control dependences in a processor implementing dynamic scheduling. If a processor supports branch prediction with dynamic scheduling, it fetches and issues instructions, as if the branch prediction was always correct. If a processor also supports hardware-based speculation, it also *executes* them.

It combines three ideas:

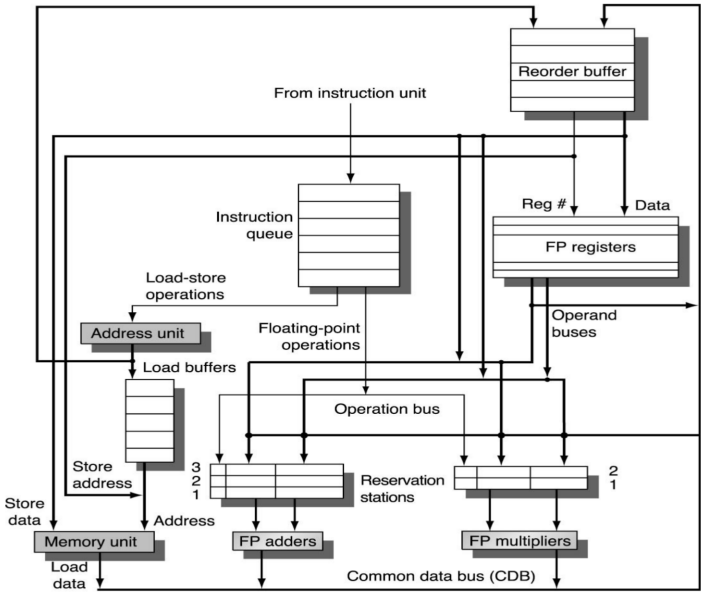
- **Dynamic branch prediction**: we assume the result of branch instructions before they finish executing;
- **Dynamic scheduling**: we schedule instructions when they are *ready* to execute;
- **Speculation**: we start executing the predicted instructions before the branch finishes executing..

In such a way, the processor implements **data flow execution**: operations execute as soon as their operands are available.

The basic Tomasulo's architecture is adopted, extending it to support speculation. There are two different steps in instruction execution:

- The computation of results and their bypassing to other instructions;
- The update of register file and memory, which is only performed when the instruction is no longer speculative (*instruction commit* via the **ROB** (*ReOrder Buffer*)); in this way, in-order commitment is implemented.

The ROB provides additional virtual registers and integrates the store buffer existing in the original Tomasulo's architecture.



8.1.1 Fields

Each ROB record has 4 *fields*:

- Instruction type: branch, store, or register;
- Destination: register number, or memory address;
- Value: contains the value when the instruction has completed but still didn't commit to a register file;
- Ready: indicates whether the instruction completed its execution.

With speculation, data for an incoming instruction may be read:

- From the ROB, if the producing instruction didn't commit yet (because a branch wasn't confirmed);
- From the register file, otherwise (this is the normal execution path we saw this far).

8.1.2 How does it work?

An instruction is extracted from the instruction queue (that is, **issued**) if there is:

- An empty reservation station;
- An empty slot in the reorder buffer.

If this is not the case, the instruction issue is stalled. Otherwise:

- The operands for the instruction are sent to the reservation station, if they are in the register file or in the reorder buffer.
- The number of the reorder buffer entry for the instruction is sent to the reservation station to tag the instruction (and its results, when they will be written on the **CDB** (*Common Data Bus*)).

At this point we want to know when this instruction will be **executed**. The instruction is executed as soon as all the required operands are available: this avoids any RAW hazard. Operands are possibly taken from the CDB as soon as another instruction pro-

duces them. The length of this step of course varies depending on the instruction type (and thus the execution unit that gets the instruction).

Next it's time for **writeback**. As soon as it is available, the result is put on the CDB (together with the tag identifying the instruction) and sent to the Reorder Buffer. Any reservation station waiting for the result reads it, and the reservation station entry is marked as available

Why does this ensure control hazards are avoided? The Reorder Buffer is ordered according to instructions original order. As soon as one instruction reaches the head of the buffer;

- If it is a mispredicted branch, the buffer is flushed, and the execution is restarted with the correct successor of the instruction;
- Otherwise, the result is written in the register file or in memory (in case of a store);
- In both cases, the Reorder Buffer entry is marked as free. The Reorder Buffer is implemented as a circular buffer.

The result of this is:

- **WAR** and **WAW** name dependencies don't result in hazards, since dynamic renaming is implemented and memory updating occurs in-order (i.e., when a store reaches the top of the ROB).

  Register renaming is implemented via the **RAT** (*Register Alias Table*) which assigns a physical register to a logical one. The RAT is updated every time an instruction is committed to a register.
- **RAW** hazards in memory are prevented by:
  - Enforcing the program order while computing the effective address of a load / store;
  - Preventing a load to initiate its second step if any active ROB entry occupied by a store has a Destination field matching the A field of the load.

  RAW hazards in registers are of course prevented by the Tomasulo architecture itself.

9 Lesson 23-10-26

So far we introduced a number of processor architecture aimed at lowering **CPI** (*Cycles Per Instruction*).

- One example was the **pipeline**, which leveraged **ILP** (*Instruction-Level Parallelism*) to increase the number of instructions taken into the execution unit at each cycle;
- We then noticed pipelines benefitted for some reordering of instructions, aimed to reduce hazards preserving dependencies, called **scheduling**. We saw schedulilng of either the static (think IA64) or the dynamic (*Tomasulo*) type;
- We then extended the Tomasulo architecture, by introducing the **ROB** (*ReOrder Buffer*), and thus educing the effects of data and control dependencies (by allowing dynamic branch prediction, and *speculation* acting on those predictions).

We know want to focus on *superscalar* architectures, able to finish more than one instruction at each clock cycle.

9.1 Multiple-issue processors

The techniques for reducing the effects of data and control dependencies can be further exploited to obtain a CPI less than one: this requires issuing more than one instruction per clock cycle. There are two kinds of processors able to do so:

- **Superscalar** processors, either statically or dynamically scheduling instructions;
- *Very Long Instruction Word* (**VLIW**) processors.

In both groups the architecture obviously includes multiple functional units.

9.1.1 Static multiple issue

Implementing a superscalar processor able to possibly issue several instructions according to the static order defined by the compiler is rather easy and inexpensive.

For instance, two instructions can be issued per clock cycle if:

- One is a load, store, branch, or integer ALU operation;
- The other is any FP operation (but load and store, which belong to the first category).

Two instructions (64 bits) are fetched and decoded at every clock cycle. The two instructions are aligned on a 64-bit boundary and constitute an *issue packet*.

Old superscalar processors required a fixed structure for issue packets (e.g., that the integer instruction is always the first one). Current processors normally do not have this limitation.

9.1.2 Dynamic multiple issue

In dynamic multiple issue, it's the processor's job to decide how to schedule instructions. Multiple issue in this case just means taking the modified Tomasulo architecture and allowing for multiple instructions to be dispatched at once.

Similarly, at this point, the commit unit will be able to take more than one instruction out of the reorder buffer at any given cycle, and commit them together.

One complicaton emerges from branches: in generale, whether we allow or not for speculation, it is best to have branch instruction dispatched alone at any given cycle.

9.2 Caches

Modern processors heavily rely on **caches** to speed up execution. Caches exploit *locality* of data and code to vastly reduce the time losses due to memory accesses.

At the most general level:

- If an entire block containing a referenced memory location is loaded in cache at time t_0;
- The program will likely find the rest of the memory it needs within the block for a time Δt.

There are 2 types of locality:

- **Temporal** locality: the same data is used more often if it was recently referenced. Within our previous framework, if at time t the progarm accesses a block of memory, it will likely access it again until $t + \Delta t$;
- **Spatial** locality: data around the data which was just used is more likely to be used next. Within our previous framework, if at time t the program accesses a block of memory at address x, it will likely also access memory at $x \pm \Delta x$.

Cache performance is given by the average *access time* in function of the hit rate h:

$$t_{ave} = h \times C + (1 - h) \times M$$

where C is the cache access time (on cache *hit*), and M is the memory access time (on cache *miss*).

10 Lesson 27-10-26

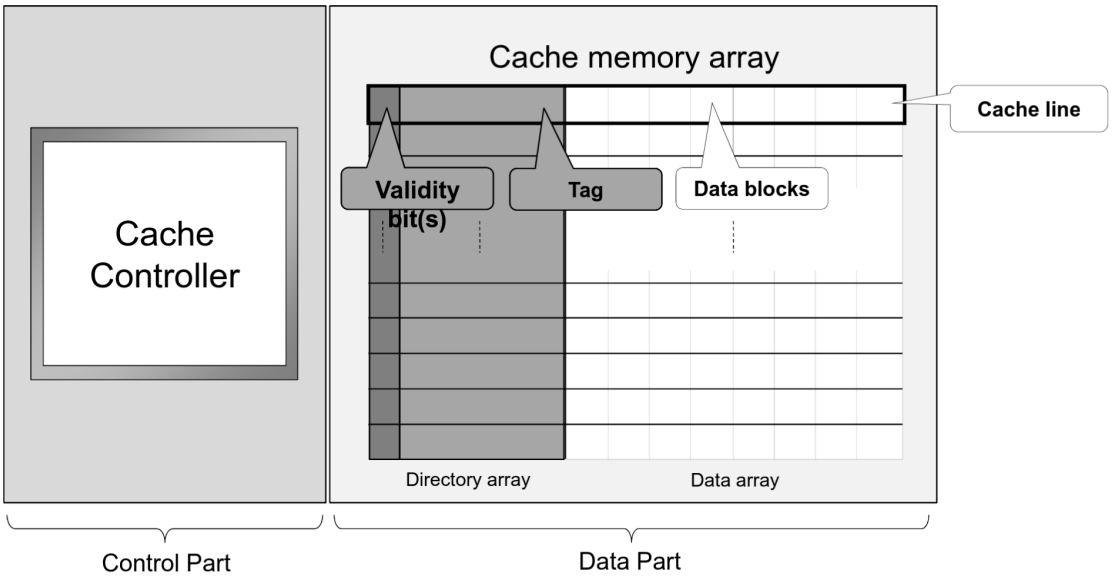
Last lesson we introduced caches.

10.0.1 Direct mapped cache

We know look at how cache memories are actually realized in practice.

The first design design is the usual *direct mapped* cache:

- A cache is organized in **cache lines**;
- A line contains a memory block that includes some memory words;
- Each line is associated with a tag field, which indicates the memory block present in the line at that time.



We also need some **control logic** for:

- Intercepting the addresses produced by the processor;
- Checking inside the cache for the possible presence of the block that the processor wants to access;
- Possibly loading the block.

As we said, the mechanism of splitting the memory address in **tag**, **index** and **offset** is the usual direct mapping technique.

- The *advantage* of it is that the mechanism can be easily implemented in hardware (the least significant block identifier bits identify the cache line)
- The *disadvantage* is that if the program frequently accesses 2 blocks corresponding to the same cache line, a miss occurs at each memory access.

10.0.2 Fully associative mapping

This is the opposite approach of direct mapping: **fully associative mapping** (rarely implemented in practice). It is the case in which each block of the main memory can be stored in any cache line.

- The *advantage* is the maximum flexibility in choosing the cache line to use;
- The *disadvantag* is the complexity of search hardware (usually adopting associative memory such as **CAM**, *Content Addressable Memory*).

10.0.3 Set associative mapping

This is an implementation which expands on direct mapping, giving some of the benefits of associative mapping: **set associative mapping**.

The cache lines are subdivided into a number S of sets, consisting of W (ways) lines. We have that:

$$S = \frac{N}{W}$$

where N is the total number of cache lines: this means that our S sets map to our old memory blocks in the direct mapping cache, and each way is basically superimposed on the relative set.

A memory block i is associated with the set k, with:

$$k = i \,\%\, S$$

same as in the direct mapping sense, but the block i can be put into any of the W lines (ways) of the set k (some kind of associative memory should be used).

A set associative cache with W lines in each set is called a W-ways cache. Common values for W are the first powers of two.

Since, as we said, the memory block can be put into any of the ways, we need mechanisms to *retrieve* the correct block and *replace* the correct block on eviction:

- **Retrieval** is easy if the number of ways is limited: we can simply use W comparators per set, each checking on a specific way;
- **Eviction** can be done via a number of algorithms, mostly ideal with heuristic implementations. The eviction algorithm defines which cache line should be used to store a memory block, amongst those associated with the block (in the case of associative or set associative mapping). It is chosen from:
  - **LRU** (*Least Recently Used*): the most used;
  - **FIFO** (*First-In First-Out*): the cheapest;
  - **LFU** (*Least Frequently Used*): theoretically, the most effective;
  - Random: simple and effective.

10.0.4 Write policy

An important policy on the memory controller which must be chosen is **write policy**. 2 main options:

- **Write-back**: when a write is done on a memory location present in a cacheline, we only update the cache memory: the changes are *written back* to DRAM only at a later moment. Very quick and efficient, bad for parallel architectures (information goes out of sync);

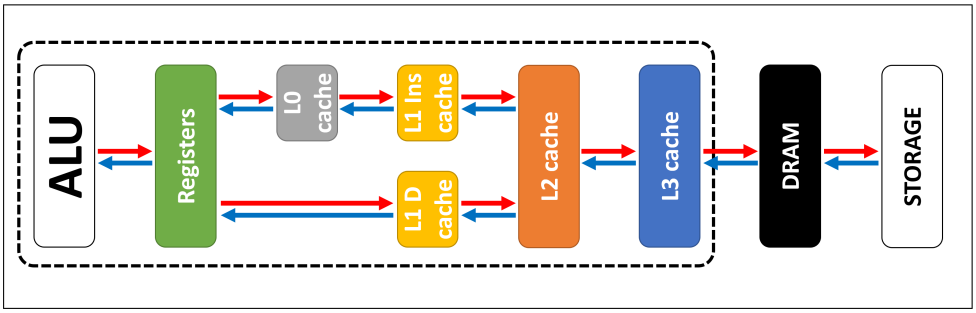
- **Write-through**: when a write is done on a memory location present in cache, both the cacheline and DRAM are updated. Cache is always synchronized to DRAM, but this option is slower.

11 Lesson 30-10-26

11.0.1 Cache levels

We talked for a while about cache memories, but not about how caches are actually organized on the local bus.

We find that they are usually close to the *processors*, organized in **levels** progressively farther from it. Usually, at level 1 only, we have a split data and instruction cache.



t may be convenient to have multiple levels of caches, as we can have:

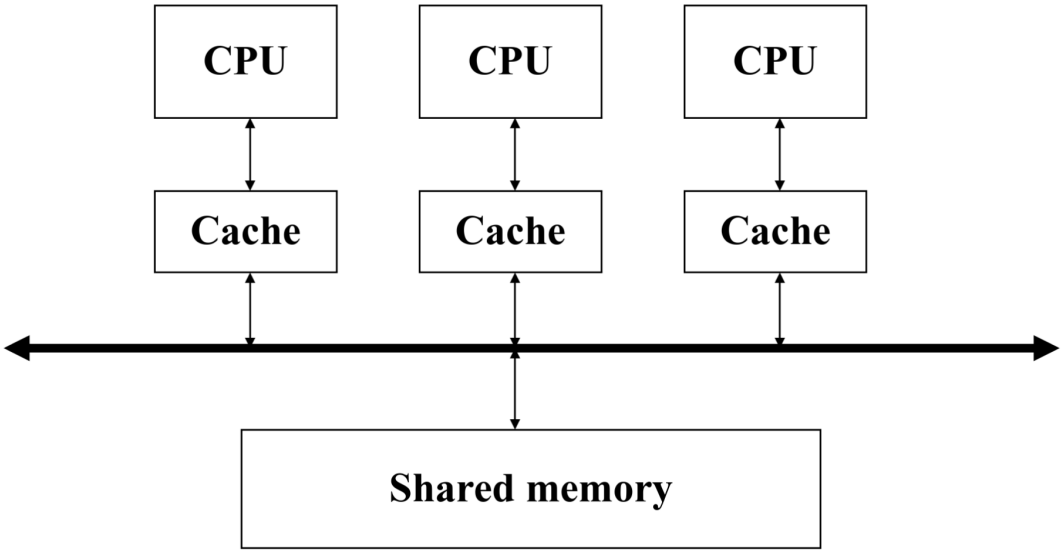
- A first level cache (L1), smaller and faster;
- A second level cache (L2), slower but larger;
- A third level cache (L3), even slower and bigger.

Each time the processor performs a memory access

1. It checks first whether the word is in L1
2. If so, it can access the word in L1
3. If not, it checks whether the word is in L2
4. If so, it will access to the word in L2 and eventually update L1
5. If not, it checks whether the word is in L3
6. If yes, L3 is accessed and L2 is updated
7. If not, it accesses to the main memory and eventually updates L3

11.0.2 Cache coherence

For multiple processor CPUs, then, we are intersted in *cache coherence* between multiple processor caches. The architeture is usually as follows:



The problem is that of course we don't know if all CPUs get the same value in their cache: even if we implement a *write through* (see 12.0.4) policy, the other caches on the bus don't notice the value change if they don't explicitly go and check if they have the updated value in their cachelines.

This is a major problem in multiprocessor systems with shared memory, in which each processor has its own cache. Similar problems may occur if the system uses a DMA controller.

To fix this and achieve cache coherence, two main approaches exist:

- **Snooping** Snooping is a process where the individual caches monitor address lines for accesses to memory locations that they have cached. If a write is detected in one of the locations contained in cacheline, the cacheline on the cache is invalidated;
- **Directory-based** In a directory-based system, the data being shared is placed in a common directory that maintains the coherence between caches. The directory acts as a filter through which the processor must ask permission to load an entry from the primary memory to its cache. When an entry is changed, the directory either updates or invalidates the other caches with that entry. The directory is usually stored in shared memory.

11.1 Branch prediction

We have somewhat hinted at **branch prediction** in the past lessons, sticking to simple predict taken or predict untaken solutions for pipelined processors. Since we have studied the Tomasulo architecture and its application insSuperscalar processors with ROBs that need flushing on misses (which is a very expensive operation), we know want to focus on more advanced techniques for predicting wether a branch is taken.

To summarize, branches can potentially impact in a very serious way the pipeline performance. It is possible to reduce the performance losses by predicting how branches will behave.

Branche prediction *schemes* aim at correctly forecasting branches, thus reducing the chance that control dependences cause stalls. They can be categorized in two groups:

- **Dynamic techniques**: they are implemented by the hardware based on the behavior of the code;

- **Static techniques**: they are handled by the compiler resorting to a preliminary analysis of the code.

This is akin to the distinction we made in 6.2 regarding ILP dynamic scheduling techniques.

11.1.1 Static branch prediction

Of course, as we said earlier, **static** branch prediction can be useful when combined with other static techniques, such as:

- Enabling delayed branches;
- Rescheduling to avoid data hazards.

The compiler may predict branch behavior in different alternative ways:

- Always predicting branches as taken: this is the simple *predict taken* scheme.
  It is generally inaccurate, with benchmarks giving anything from 5% to 60% misprediction rate;
- Predicting branch behavior depending on branch direction, which is a simple but highly accurate techniques for code in standard form (such as code produced by C compilers).
  The technique is based on the observation that, mainly due to loop constructs in high level languages:
  - Forward branches are more often untaken: the only real cases in high level code is the `goto` statement sometimes used in kernel-level C code to simulate exception handling;
  - Backward branches are more often taken: these are the common branches inside `while` and `for` loops. If a loop repeats n times, then $n-1$ times the branch is taken, thus the heuristic.
- Predicting on the basis of **profile** information coming from earlier runs. Note that this is *not* dynamic yet (!). We are making heuristic decisions on profile data collected at compile time, which then gets hardcoded.
  So, the technique is based on:
  1. Identifying a typical sequence of input stimula for the program;
  2. Running the program a limited number of times using these stimula;
  3. Gathering statistics on the behavior of branches;
  4. Using these statistics as predictions.

11.1.2 Dynamic branch prediction

Dynamic schemas are based on hardware, and use the branch instruction address to activate the different prediction mechanisms. Dynamic branch prediction can be based on different techniques:

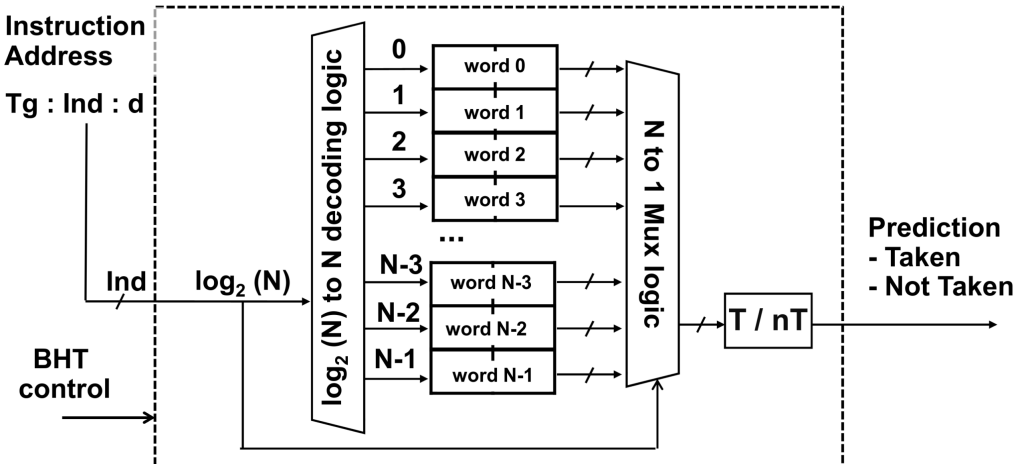
- *Branch History Table* (**BHT**): one- and two-bit prediction schemes.
  It is the simplest method for dynamic branch prediction. The BHT is a small memory:
  - Indexed by the lower portion of the address of the branch instruction (same as in cache direct mapping);

– Containing for each entry one or more bits recording whether the branch has been taken or not the last time it has been executed.

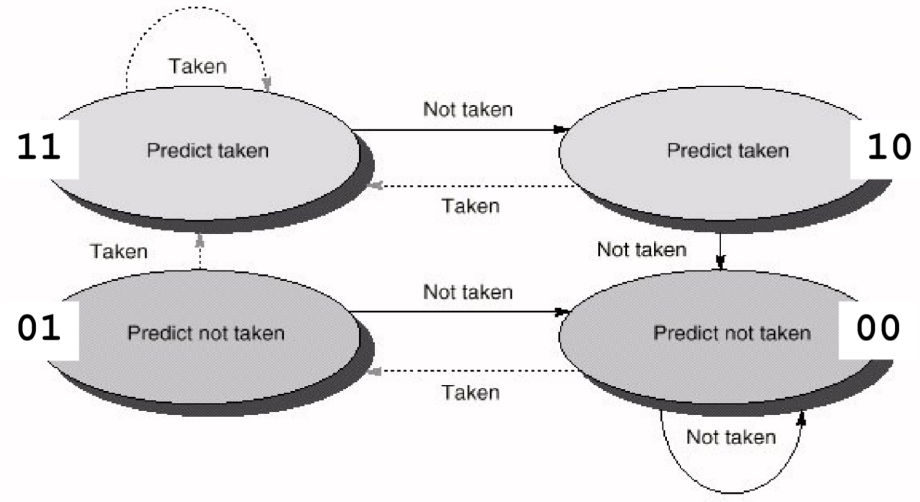
Each time a branch instruction is decoded:

1. An access is made in the BHT using the lowest portion of its address as an index
2. The prediction stored in the table is used, and the new PC is computed according to the prediction.
3. When the branch result is known, the BHT is possibly updated.

A BHT using only **one bit** at every table location is called a *Bimodal* predictor



Two bit BHT entries can also be used. They provide higher prediction capabilities. If for every branch, two bits are maintained, the prediction is changed only after missing twice.



- Two-level prediction schemes (correlating predictors);
- *Branch Target Buffer* (**BTB**);
- Others.

12 Lesson 03-11-26

During the last lesson we talked about *branch prediction tables* used for *dynamic branch prediction*.

We move on to other schemas now.

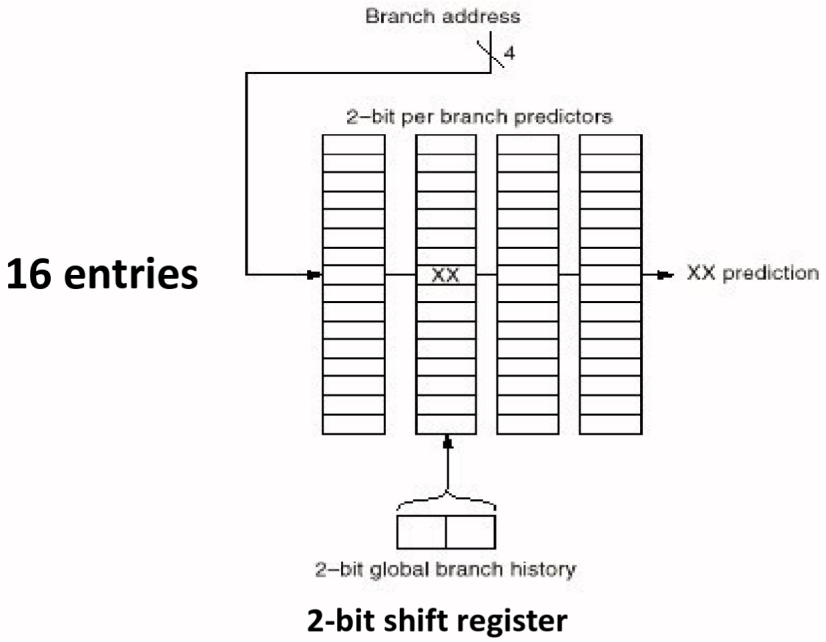
- **Correlating predictors** (also known as *Bilevel* predictors): this approach is based on the dependencies between the results of last branches. What we mean is that the behavior of a branch is strongly dependent on the result of the previous ones.

  (m, n) predictors use the behavior of the last m branches to choose from 2^m branch predictors, each of which is a n-bit predictor. The hardware required for implementing this scheme is very simple:

  – The history of the most recent m branches is recorded in an m-bit shift register, where each bit records whether the branch was taken or not;
  – The branch-prediction buffer is indexed using a concatenation of the low-order bits from the branch address with the m low-order history bits.

  Let's see a few examples:

  – $(1, 1)$ predictor: in this case, $m = 1$ and $n = 1$. Each branch is associated with 2 predictors of one bit: one reporting the prediction in the case the previous branch was taken, one with the prediction in the case the previous branch was not taken;
  – $(2, 2)$ predictor: same as above, but with $m = 2$ and $n = 2$. Each branch is associated with 4 predictors of 2 bits.



12.0.1 Branch-Target Buffer

The **BTB** (*Branch Target Buffer*) reduces an additional delay introduced by branch instructions: the fact that we may not know the delay target.

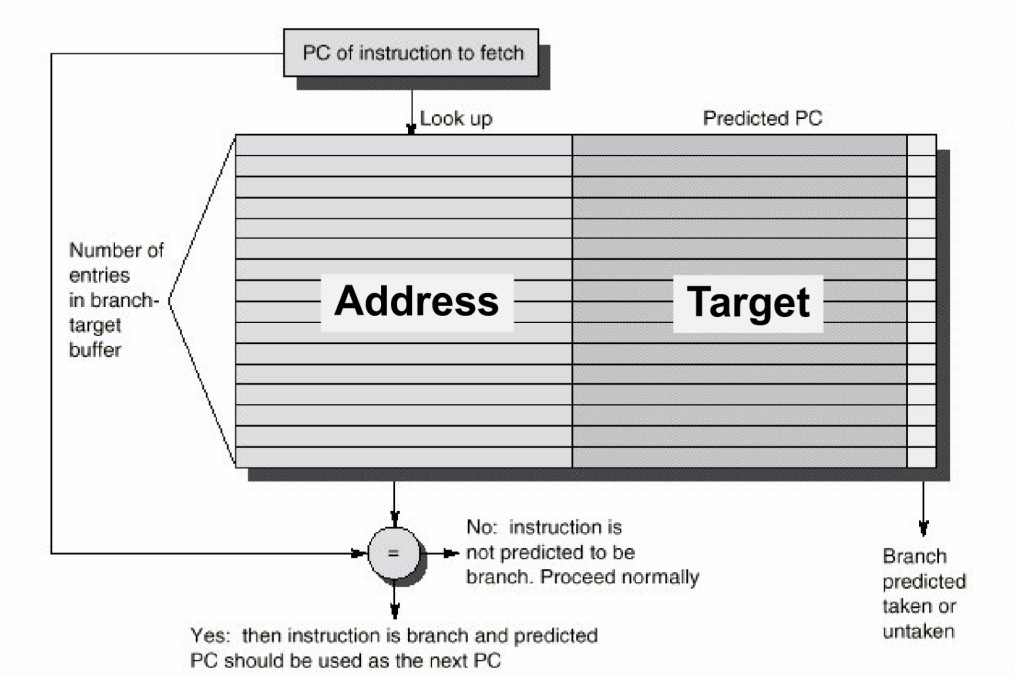
Reducing the negative effects of control dependencies requires knowing as soon as possible both:

- Whether the branch has to be taken or not;
- The new value of the PC (if the branch is assumed to be taken).

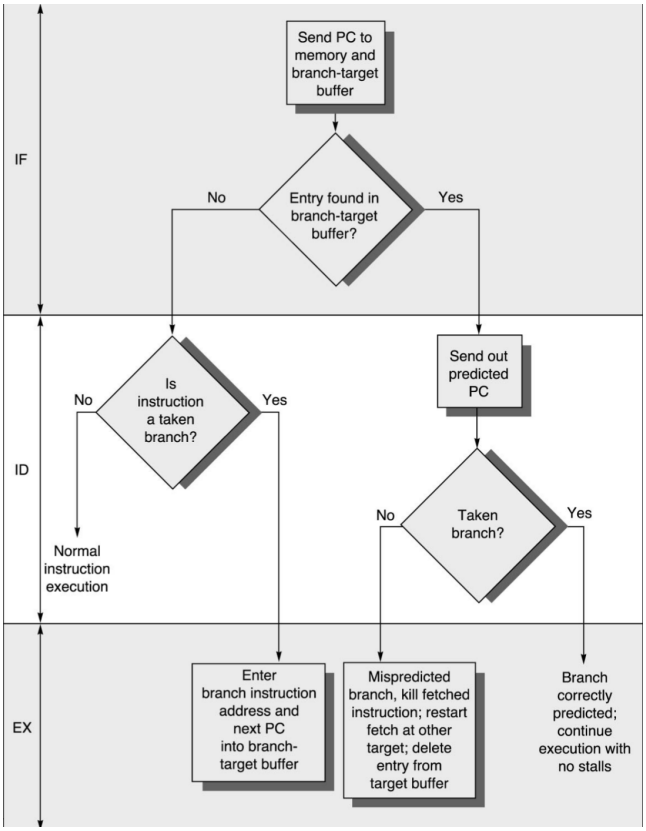
This later issue is faced by introducing a *branch-target buffer* (or cache). Each entry of the branch-target buffer contains:

- The address of the considered branch;
- The target value to be loaded in the PC.

Using the branch-target buffer, the PC is loaded with the new value at the end of the IF stage, i.e., even before the branch instruction is decoded.



We have a flowchart that shows how we interact with the BTB based on the execution phase of a branch instruction we find ourselves in:



13 Lesson 09-10-26

13.1 Introduction to ARM architectures

ARM (which means anything from *Acorn RISC Machine* to *Advanced RISC Machine*) is a british company specialized in RISC processor designs (sold as **IP**, *Intellectual Property*).

13.1.1 Timeline

A short timeline is:

- 1985: ARM1, first ARM processor (v1), designed by Acorn Computers produced by VLSI Technology (now NXP);
- 1987-1989: ARM2 and ARM3 (v2), used by Acorn in its desktop PCs, sold by VLSI as part of processor chip sets;
- 1990: Acorn and Apple found ARM Ltd. with a new business model: manufacturing rights instead of processors. Of course, VLSI become the first licensee;
- 1991: ARM6 (v3);
- 1993: ARM7 (v4), ARM7TDMI: Debug and ICE (In-Circuit Emulation) plus a new compressed instruction set: **Thumb**;
- 1996-2006: ARM8 - ARM11 (v5, v6);
- 2004-today: Cortex cores (v7, v8, v9).

Today the 3 main product lines mantained by ARM are:

- **Cortex-A, X**: high-end applications (smartphones)
- **Cortex-R**: real-time and safety-critical applications

- **Cortex-M**: microcontrollers (Cortex-M3 LPC1768)

13.1.2 Business model

ARM has seldom delivered stand-alone devices with its own microprocessor. It mainly sells cores, to be used for integration in microcontrollers or *Systems on Chip* (**SoC**s).

Two types:

- **Hard cores**: ARM provides a physical layout optimized for a specific semiconductor technology
- **Soft cores**: ARM provides a high-level description that the designer can synthesize for a specific technology.

13.1.3 Microprocessor vs microcontrollers

Let's look at the difference between **microprocessors** (which we are used to) and **microcontrollers**. A microcontroller is a single computer system packed into a computer chip, which can in term be placed in a board or some other device, to control it as an *embedded system*.

A common format is **SoC**, which we said means *System-on-Chip*. An SoC is fundamentally an entire system integrated in a single piece of silicon. Most microcontrollers are examples of SoC integration technologies, but SoCs are widely used in other applications too.

To summarize:

- Microprocessor describes a processor-oriented chip;
- Microcontroller describes a computer optimized for control, typically with integrated memory and peripherals;
- SoC describes the integration of a system's major components into one chip.

13.1.4 ARM v7-M architecture

Let's give a quick overview of the ARM v7-M architecture, which is the architecture used in the ARM Cortex-M3 processor we will be using.

Data types The following data types are supported:

- byte: 8 bits
- halfword: 16 bits
- word: 32 bits

General registers Registers are 32-bit wide.

We have the following general-purpose registers (some aliased):

Register	Synonym	Use
r0	a1	general purpose registers
r1	a2	
r2	a3	
r3	a4	
r4	v1	
r5	v2	
r6	v3	
r7	v4	
r8	v5	
r9	v6, SB	
r10	v7	
r11	v8	
r12	IP	
r13	SP, MSP, PSP	stack pointer
r14	LR	link register
r15	PC	program counter

We sse the aliases in the table. The main ones are:

- **IP**: not really significant, means *Intra-Procedure scratch* for compiler generated routines (see AAPCS below);
- **SP**: this is the *Stack Pointer*, with **MSP** and **PSP** being respectively the *Main* (handler) and *Program* (thread/user) stack pointers in the respective modes;
- **LR**: this is the *Link Register* for jump and link calling convention;
- **PC**: the good old program counter.

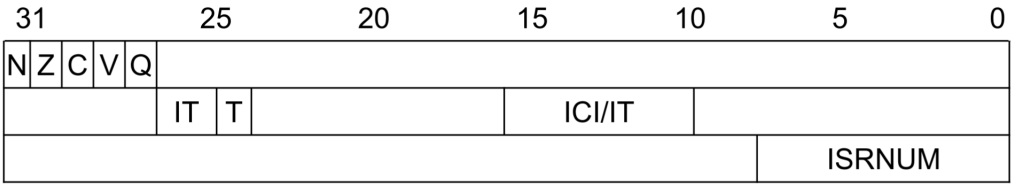
Arm defines **AAPCS** (*ARM Architecture Procedure Call Standard*) for the caller convention ABI. A quick overview is:

- Registers from `r0` to `r3` are the first 4 function arguments (from the fifth onwards we use the stack). Register `r0` then takes the return value. These are *caller-saved*;
- Registers from *r4* to *r11* are *callee-saved* scratch;
- Register `r12` is, as we said, a special *intra-procedure scratch*. The only significant characteristic for now is that it's *caller-saved*, as the first four.

Special registers We also have the following special registers:

Special register	Use
xPSR	program status register
PRIMASK	interrupt mask registers
FAULTMASK	
BASEPRI	
CONTROL	control register

The **PSR** (*Program Status Register*) is significant.



It can be accessed all at once (PSR) or as a combination of 3 registers (in order, the three rows of the figure above):

- *Application Program Status Register* (**APSR**): This contains application info (akin to the status flags on x86):
  - N: Negative result from ALU;
  - Z: Zero result from ALU;
  - C: ALU operation Carried out;
  - V: ALU operation oVerflowed;
  - Q: "sticky" flag for saturation arithmetic.
- *Execution Program Status Register* (**EPSR**). This contains program execution status usually not of interest to the programmer:
  - IT: IF-THEN instruction status bits. This is interesting: the ARMv7-M (Cortex-M3) uses conditional execution to let instructions execute only when a condition is true. The IT instruction controls this for Thumb instructions;
  - ICI: Interrupt-Continuable Instruction bits;
  - T: Thumb bit. It is always 1: clearing this bit will cause a hard fault exception in ARM-v7.
- *Interrupt Program Status Register* (**IPSR**). It contains an exception number used in exception handling. The ISRNUM is extremely important as, if it is different than 0, signals that the processor is in *handler* mode (the equivalent of system mode in ARM).

Instructions There are 3 kinds of instructions:

- ARM instructions: 32 bits wide
- Thumb instructions: 16 bits wide
- Thumb-2 instructions: 16 or 32 bits wide

In fact, ARM v7-M architecture supports Thumb and Thumb-2 instructions only.

Processor modes We have orthogonal operating and privilege modes.
There are two operating modes:

- Thread mode: on reset or after completing the exception routine;
- Handler mode: when an exception occurs.

And two access levels:

- User level: limited access to resources;
- Privileged level: access to all resources.

Note that handler mode is always privileged.
The CONTROL register has only 2 bits, which control:

- Bit 0, which is **nPRIV**, controls the access level;
- Bit 1, which is **SPSEL**, controls the current stack (main or process).

The CONTROL register can be changed only in privileged mode. When an exception is taken, the processor automatically switches to Handler mode. When the handler finishes and exception return occurs, the processor returns to Thread mode if no other exception handler should run. This means handler state is encoded in IPSR $\neq 0$.

We show a summarizing table:

	privileged level	user level
handler mode	CONTROL[0] = 0 CONTROL[1] = 0	not allowed
thread mode	CONTROL[0] = 0 CONTROL[1] = 0/1	CONTROL[0] = 1 CONTROL[1] = 0/1

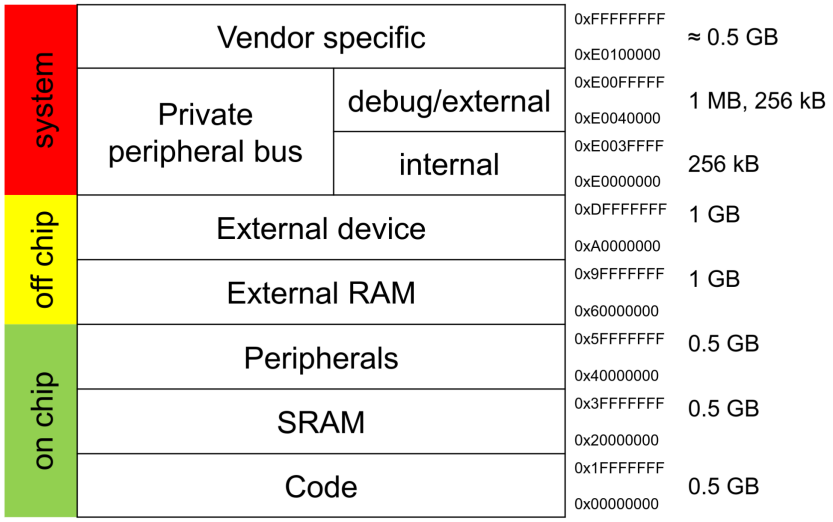
Interrupt Vector Table ARM-v7 has an **IVT** (*Interrupt Vector Table*).

- Each entry manages an exception, interrupt, or other atypical events such as a reset;
- The vector table contains either ARM instructions or addresses in memory: in the v7-M architecture, there are addresses.

14 Lesson 13-10-26

14.1 Cortex-M3 memory map

Let's look at the Cortex-M3 memory map.



We see a bunch of significant sections:

- The first region, up to `0x1FFFFFFF`, is dedicated to code (on **FLASH** memory);
- We then have **SRAM** (for data, stack and heap regions), up to `0x3FFFFFFF`;
- Lastly we have **on-chip** *peripherals*, ranging up to `0x5FFFFFFF`;
- The region from `0x60000000` to `0xDFFFFFFF` is equally split between **external RAM** and **external devices**.
- Lstly, the region from `0xE0000000` is system-reserved, dedicated to the private peripheral bus and vendor specific data / registers.

Let's see a table of the memory region access characteristics:

Region	Bufferable	Cacheable	Executable
Code	yes	write through	yes
SRAM	yes	write back	yes
Peripherals	no	no	no
External RAM	no	write back	yes
External device	no	no	no
System	no	no	no

15 Lesson 06-11-26

15.1 MOV instructions in ARM

In assembly ARM we have a number of instructions to load literals into registers, worth seeing.

- The usual `mov` is `mov <rd>, <rn> {, shift}`. `shift` is an optional shift applied to `Rn`:

  - `ASR #n` : arithmetic shift right;
  - `LSL #n` : logical shift left;
  - `LSR #n` : logical shift right;
  - `ROR #n` : rotate right;
  - `RRX` : rotate right 1 bit with extend.
- There is also a way to move a constant, with `mov <Rd>, #constant`. The constant is stored on 8 bits so there are limitations. It can be:
  - A value obtained by shifting left an 8-bit value up to 24 positions;
  - A value of the form `0x00XY00XY`;
  - A value of the form `0xXY00XY00`;
  - A value of the form `0xXYXYXYXY`.
- `mvn` has the same exact syntax as `mov`, but moves the one's complement of the operand into a register;
- `movw` is the wide equivalent of `mov`. It moves a 16-bit value in the low halfword of a register. The assembler can replace `mov` with `mvn` or `MOVW` if needed. This increases the number of possible values supplied as constants to `mov`:
  - A 16-bit value: 0-65535 (`mvw`),
  - A value obtained by shifting left an 8-bit constant up to 24 positions;
  - A value obtained by shifting left an 8-bit constant up to 24 positions, and then applying a bitwise logical NOT operation (`mvn`);
  - A value of the form `0x00XY00XY` or `0xFFXYFFXY` (`mvn`);
  - A value of the form `0xXY00XY00` or `0xXYFFXYFF` (`mvn`);
  - A value of the form `0xXYXYXYXY`.
- `movt` is the upper 16-bit version of `movw`. It moves a 16-bit value in the high halfword of a register. A register can be set to any 32-bit constant by using `movw` and `movt` together, so the pseudo-instruction `mov32` is translated into a combination of those.

15.1.1 LDR for constants

Besides loading values from memory (also note that `mov` can't load value adderesses), `LDR` can be used to load constants into registers: `ldr <Rd>, =<constant>`.

It is a pseudo-instruction:

- If constant is among the valid values of MOV, then the instruction is replaced with: `mov <Rd>, #<constant>`;
- Otherwise, a block of constants, called **literal pool**, is created and the instruction becomes: `ldr <Rd>, [PC, #<offset>]`.